

TRƯỜNG ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN THỰC TẬP CHUYÊN NGÀNH
NHẬN DẠNG CHỮ SỐ VIẾT TAY
HUẤN LUYỆN VÀ SO SÁNH KNN, SVM
TRÊN TẬP DỮ LIỆU MNIST

Giảng viên hướng dẫn: **THS. NGUYỄN NHỨT LAM**

Sinh viên thực hiện: **PHAN THÀNH THUẬN**

Mã số sinh viên: **170123591**

Lớp: **DX23TT11**

Khoá: **2023–2027**

Trà Vinh, tháng 10 năm 2026

TRƯỜNG ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN THỰC TẬP CHUYÊN NGÀNH
NHẬN DẠNG CHỮ SỐ VIẾT TAY
HUẤN LUYỆN VÀ SO SÁNH KNN, SVM
TRÊN TẬP DỮ LIỆU MNIST

Giảng viên hướng dẫn: **THS. NGUYỄN NHỨT LAM**

Sinh viên thực hiện: **PHAN THÀNH THUẬN**

Mã số sinh viên: **170123591**

Lớp: **DX23TT11**

Khoá: **2023–2027**

Trà Vinh, tháng 10 năm 2026

TÓM TẮT ĐỒ ÁN

Đồ án huấn luyện và so sánh hiệu quả của hai mô hình học máy kinh điển — K-Nearest Neighbors (KNN) và Support Vector Machine (SVM) nhân RBF — trên bài toán nhận dạng chữ số viết tay, sử dụng bộ dữ liệu chuẩn MNIST (70.000 ảnh chữ số 0-9, kích thước 28×28 điểm ảnh mức xám). Đây là đúng đề tài được phân công: tìm hiểu học máy, xây dựng mô hình và ứng dụng minh hoạ giải quyết bài toán của đề tài.

Dữ liệu được chia theo tỉ lệ 54.000/6.000/10.000 (train/validation/test) theo phương pháp stratify. Kết quả thực nghiệm thật trên tập test: KNN ($k=5$) đạt độ chính xác 97,08%, thời gian huấn luyện 0,07 giây; SVM (kernel RBF, $C=5$) đạt độ chính xác 98,35%, thời gian huấn luyện 235,17 giây.

Ngoài kết quả tổng thể, đồ án còn khảo sát thực nghiệm ảnh hưởng của tham số k và độ đo khoảng cách lên KNN, độ nhạy của tham số C và γ lên SVM, và trực quan hoá không gian đặc trưng của dữ liệu bằng PCA — cho thấy SVM vượt trội hơn KNN nhờ khả năng học ranh giới phi tuyến, đổi lại chi phí huấn luyện cao hơn nhiều bậc, đúng như giả thiết khoa học đặt ra ở Chương 2. Một chương trình demo tương tác được xây dựng bằng Streamlit, cho phép người dùng vẽ hoặc tải ảnh một chữ số lên và xem cả hai mô hình dự đoán kèm điểm tin cậy.

Từ khoá: nhận dạng chữ số viết tay, MNIST, K-Nearest Neighbors, Support Vector Machine, học máy, scikit-learn.

MỤC LỤC

MỞ ĐẦU.....	vi
1. Lý do chọn đề tài.....	vi
2. Mục tiêu nghiên cứu.....	vi
3. Đối tượng và phạm vi nghiên cứu.....	vii
4. Phương pháp nghiên cứu.....	viii
5. Cấu trúc báo cáo.....	viii
CHƯƠNG 1. TỔNG QUAN.....	1
1.1. Bài toán nhận dạng chữ số viết tay.....	1
1.2. Bộ dữ liệu MNIST.....	1
1.3. Tình hình nghiên cứu: các kết quả đã công bố trên MNIST.....	2
1.4. Các hướng tiếp cận hiện nay.....	4
1.5. Phạm vi và lựa chọn của đề án.....	5
1.6. Tổng kết chương.....	5
CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT.....	6
2.1. Bài toán phân loại có giám sát.....	6
2.2. Thuật toán K-Nearest Neighbors.....	6
2.3. Support Vector Machine.....	8
2.3.1. Siêu phẳng phân tách và bài toán lề cực đại.....	8
2.3.2. Biên mềm và tham số C.....	9
2.3.3. Kernel trick và hàm kernel RBF.....	9
2.3.4. Chiến lược phân loại đa lớp.....	10
2.3.5. Chuẩn hoá dữ liệu và ảnh hưởng tới các mô hình dựa trên khoảng cách.....	11
2.4. Các độ đo đánh giá mô hình.....	11
2.5. Công cụ, công nghệ và phần mềm sử dụng.....	13
2.6. Phân tích độ phức tạp tính toán.....	14
2.7. Giả thiết khoa học của đề án.....	15
2.8. Tổng kết chương.....	15
CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU.....	17
3.1. Quy trình tổng thể.....	17
3.2. Kiến trúc hệ thống và cấu trúc mã nguồn.....	17
3.3. Chuẩn bị dữ liệu.....	19
3.4. Cài đặt mô hình KNN.....	19
3.5. Cài đặt mô hình SVM.....	20
3.6. Quy trình huấn luyện và lưu kết quả.....	20
3.7. Thiết kế chương trình demo.....	21
3.8. Kiểm thử tự động.....	21
3.8.1. Một thách thức kỹ thuật thực tế đã gặp phải.....	21
3.9. Quản lý mã nguồn và quy trình làm việc.....	22

3.10. Tổng kết chương.....	22
CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU.....	23
4.1. Cách chia dữ liệu.....	23
4.2. Kết quả tổng thể trên tập test.....	23
4.3. Ảnh hưởng của tham số k lên KNN.....	24
4.4. Ảnh hưởng của độ đo khoảng cách lên KNN.....	25
4.5. Độ nhạy tham số C và gamma của SVM.....	26
4.6. Kết quả phân theo từng chữ số.....	28
4.7. Trực quan hoá không gian đặc trưng bằng PCA.....	30
4.8. So sánh thời gian huấn luyện.....	31
4.9. So sánh thời gian dự đoán (độ trễ).....	32
4.10. Đường cong học theo kích thước tập huấn luyện.....	33
4.11. Phân tích định tính các ca lỗi.....	34
4.12. Kết quả chương trình demo.....	35
4.13. Hạn chế của chương trình demo.....	36
4.14. Thảo luận tổng hợp.....	36
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	38
DANH MỤC TÀI LIỆU THAM KHẢO.....	40
PHỤ LỤC.....	42

MỞ ĐẦU

1. Lý do chọn đề tài

Nhận dạng chữ số viết tay là một trong những bài toán kinh điển và dễ tiếp cận nhất của thị giác máy tính, nhưng vẫn mang đầy đủ các thách thức cơ bản của học máy có giám sát: cùng một chữ số được những người khác nhau viết theo nhiều nét khác nhau, cùng một người viết cùng một số cũng không bao giờ cho hai nét vẽ giống hệt nhau.

Bài toán này còn có giá trị thực tiễn rõ ràng: đọc số trên phiếu điền tay (mã bưu điện, số trên séc ngân hàng, phiếu khảo sát, bài thi trắc nghiệm), là bước xử lý đầu vào cho nhiều hệ thống số hoá giấy tờ.

Bộ dữ liệu MNIST — do Yann LeCun và cộng sự công bố — là bộ dữ liệu chuẩn, đã được dùng để đánh giá hầu hết các thuật toán học máy và học sâu trong hơn 25 năm qua. Việc huấn luyện và so sánh hai thuật toán học máy kinh điển (KNN, SVM) trên MNIST phù hợp với quy mô một đề án thực tập chuyên ngành: có đủ độ khó để thể hiện hiểu biết về học máy, nhưng không đòi hỏi phần cứng GPU hay thời gian huấn luyện quá dài.

Đây là đúng đề tài được phân công: "Huấn luyện và so sánh hiệu quả của mô hình học máy KNN và SVM trên tập dữ liệu MNIST", với mục tiêu tìm hiểu học máy và xây dựng một ứng dụng minh hoạ hoạt động được. Báo cáo này trình bày đầy đủ kết quả thật thu được từ việc thực hiện đề tài đó, kèm theo các thực nghiệm khảo sát sâu hơn về ảnh hưởng của tham số và đặc tính dữ liệu.

2. Mục tiêu nghiên cứu

2.1. Mục tiêu chung

Tìm hiểu về học máy; xây dựng và huấn luyện hai mô hình học máy (KNN và SVM) giải quyết đúng bài toán nhận dạng chữ số viết tay trên bộ dữ liệu MNIST; so sánh hiệu quả của hai mô hình; xây dựng một ứng dụng minh hoạ hoạt động được, có dữ liệu thật để trình diễn.

2.2. Mục tiêu cụ thể

- Nắm vững nguyên lý hoạt động của thuật toán K-Nearest Neighbors và Support Vector Machine (kernel RBF), bao gồm cơ sở toán học và các tham số điều khiển.
- Tải và chuẩn bị bộ dữ liệu MNIST thật, chia tập train/validation/test theo phương pháp stratify.
- Cài đặt, huấn luyện cả hai mô hình bằng scikit-learn trên cùng một tập dữ liệu, cùng một cách chia, để so sánh công bằng.
- Đánh giá hai mô hình bằng các độ đo chuẩn, khảo sát ảnh hưởng của các siêu tham số chính (k, độ đo khoảng cách, C, gamma) bằng thực nghiệm thật, không suy đoán.
- Xây dựng chương trình demo bằng Streamlit: người dùng vẽ hoặc tải ảnh một chữ số lên, xem cả hai mô hình dự đoán.

3. Đối tượng và phạm vi nghiên cứu

3.1. Đối tượng nghiên cứu

- Ảnh chữ số viết tay dạng số hoá, mức xám, kích thước chuẩn 28×28 điểm ảnh (định dạng của bộ dữ liệu MNIST).
- Hai thuật toán học máy có giám sát: K-Nearest Neighbors và Support Vector Machine.

3.2. Phạm vi và các lựa chọn của đề án

Phạm vi và các lựa chọn của đề án

Khía cạnh	Lựa chọn của đề án	Lý do
Bài toán	Phân loại đa lớp (10 lớp, chữ số 0-9)	Đúng bài toán MNIST chuẩn, có nhãn thật để đánh giá
Bộ dữ liệu	MNIST thật (70.000 ảnh, qua <code>sklearn.datasets.fetch_openml</code>)	Bộ dữ liệu chuẩn của ngành, công nghệ gợi ý trong đề tài là Scikit-learn

Mô hình	KNN và SVM (kernel RBF)	Đúng hai mô hình đề tài yêu cầu so sánh
Công cụ	Python, scikit-learn, Streamlit	Đúng công nghệ gợi ý trong đề tài
Đầu ra demo	Nhân dự đoán (0-9) kèm điểm tin cậy của cả hai mô hình	Minh hoạ trực quan sự khác biệt giữa hai mô hình

(Nguồn: đề tài được phân công)

3.3. Những nội dung không thực hiện

- Các mô hình học sâu (CNN, mạng nơ-ron nhiều lớp) — đề tài chỉ yêu cầu KNN và SVM; TensorFlow/Keras được đề tài gợi ý như một lựa chọn công nghệ, không bắt buộc dùng.
- Nhận dạng chữ số viết tay trên ảnh chụp thực tế có nền phức tạp, nhiều số liên nhau (ngoài phạm vi: đề tài chỉ xét từng chữ số đơn lẻ, giống định dạng MNIST).
- Triển khai sản phẩm thành dịch vụ thực tế quy mô lớn.

4. Phương pháp nghiên cứu

Đồ án sử dụng phương pháp thực nghiệm: cài đặt và huấn luyện thật hai mô hình bằng scikit-learn trên dữ liệu MNIST thật, đo đạc thật các độ đo đánh giá trên một tập kiểm thử tách biệt hoàn toàn với tập huấn luyện, và khảo sát thực nghiệm (ablation study) ảnh hưởng của các siêu tham số chính. Toàn bộ số liệu trong Chương 4 được đọc trực tiếp từ các tệp kết quả thật (`results/comparison\_summary.json`, `results/extra\_experiments.json`) sinh ra bởi `scripts/train.py` và `scripts/extra\_experiments.py`, không dùng số liệu giả định.

5. Cấu trúc báo cáo

Ngoài phần Mở đầu và Kết luận, nội dung báo cáo gồm 4 chương, theo đúng bố cục quy định:

- Chương 1 — Tổng quan: bài toán, bộ dữ liệu MNIST, tình hình nghiên cứu đã công bố, các hướng tiếp cận, phạm vi đề án.
- Chương 2 — Nghiên cứu lý thuyết: cơ sở lý thuyết của KNN và SVM, các độ đo đánh giá, công cụ/công nghệ sử dụng, giả thiết khoa học.
- Chương 3 — Hiện thực hóa nghiên cứu: kiến trúc hệ thống, các bước cài đặt, thiết kế chương trình demo, kiểm thử.
- Chương 4 — Kết quả nghiên cứu: kết quả thật, các thực nghiệm khảo sát tham số, phân tích lỗi, kết quả demo.
- Chương 5 — Kết luận và hướng phát triển: tổng kết kết quả đạt được, hạn chế, đề xuất hướng phát triển.

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn ThS. Nguyễn Nhứt Lam đã tận tình định hướng và hỗ trợ em trong suốt quá trình thực hiện đồ án thực tập chuyên ngành này.

Em cũng xin cảm ơn quý thầy cô KHOA CÔNG NGHỆ THÔNG TIN, TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ, TRƯỜNG ĐẠI HỌC TRÀ VINH đã truyền đạt kiến thức nền tảng về học máy trong suốt quá trình học tập, là cơ sở để em có thể tiếp cận và triển khai đồ án ở mức độ kỹ thuật như trình bày trong báo cáo này.

Em xin chân thành cảm ơn.

Phan Thành Thuận

NHẬN XÉT

(Của cơ quan thực tập, nếu có)

Đồ án này được thực hiện hoàn toàn trong phạm vi học thuật tại trường (xây dựng và huấn luyện mô hình học máy trên dữ liệu công khai), không có giai đoạn thực tập tại một cơ quan/doanh nghiệp ngoài trường, nên không có nhận xét của cơ quan thực tập cho mục này.

NHẬN XÉT

(Của giảng viên hướng dẫn trong đồ án của sinh viên)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Trà Vinh, ngày tháng năm

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

UBND TỈNH TRÀ VINH

**CỘNG HOÀ XÃ HỘI CHỦ NGHĨA
VIỆT NAM**

TRƯỜNG ĐẠI HỌC TRÀ VINH

Độc lập – Tự do – Hạnh phúc

BẢN NHẬN XÉT ĐỒ ÁN THỰC TẬP CHUYÊN NGÀNH

(Của giảng viên hướng dẫn)

Họ và tên sinh viên: Phan Thành Thuận

MSSV: 170123591

Ngành: Công nghệ thông tin

Khoá: 2023–2027

Tên đồ án: NHẬN DẠNG CHỮ SỐ VIẾT TAY HUẤN LUYỆN VÀ SO SÁNH
KNN, SVM TRÊN TẬP DỮ LIỆU MNIST

Họ và tên Giáo viên hướng dẫn: ThS. Nguyễn Nhứt Lam

Chức danh:

Học vị: Thạc sĩ

NHẬN XÉT

1. Nội dung đồ án:

.....
.....

2. Ưu điểm:

.....
.....

3. Nhược điểm:

.....
.....

4. Điểm mới đồ án:

.....
.....

5. Giá trị thực trên đồ án:

.....

.....

7. Đề nghị sửa chữa bổ sung:

.....

.....

8. Đánh giá:

.....

.....

Trà Vinh, ngày tháng năm 20.....

Giảng viên hướng dẫn

(Ký & ghi rõ họ tên)

NHẬN XÉT

(Của giảng viên chấm trong đồ án của sinh viên)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Giảng viên chấm
(ký và ghi rõ họ tên)

UBND TỈNH TRÀ VINH

**CỘNG HOÀ XÃ HỘI CHỦ NGHĨA
VIỆT NAM**

TRƯỜNG ĐẠI HỌC TRÀ VINH

Độc lập - Tự do - Hạnh phúc

BẢN NHẬN XÉT ĐỒ ÁN THỰC TẬP CHUYÊN NGÀNH

(Của cán bộ chấm đồ án)

Họ và tên người nhận xét:

Chức danh:

Học vị:

Chuyên ngành:

Cơ quan công tác:

NHẬN XÉT

1. Nội dung đồ án:

.....
.....

2. Ưu điểm:

.....
.....

3. Khuyết điểm:

.....
.....

4. Đề nghị sửa chữa bổ sung:

.....
.....

5. Đánh giá:

.....

.....
Trà Vinh, ngày tháng năm 20.....

Giảng viên chấm

(Ký & ghi rõ họ tên)

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải nghĩa
KNN	K-Nearest Neighbors — K láng giềng gần nhất
SVM	Support Vector Machine — Máy vector hỗ trợ
RBF	Radial Basis Function — Hàm cơ sở xuyên tâm (một loại kernel)
MNIST	Modified National Institute of Standards and Technology database — bộ dữ liệu chữ số viết tay chuẩn
PCA	Principal Component Analysis — Phân tích thành phần chính
OvR/OvO	One-vs-Rest / One-vs-One — hai chiến lược mở rộng bộ phân loại nhị phân cho bài toán đa lớp
TP/TN/FP/FN	True Positive/True Negative/False Positive/False Negative
CPU/GPU	Central/Graphics Processing Unit — Bộ xử lý trung tâm/đồ họa

CHƯƠNG 1. TỔNG QUAN

GVHD: ThS. Nguyễn Nhứt Lam

SVTH: Phan Thành Thuận

1.1. Bài toán nhận dạng chữ số viết tay

Cho một ảnh chứa một chữ số viết tay (0-9), hệ thống phải xác định đúng chữ số đó. Đây là bài toán phân loại đa lớp (multi-class classification) với 10 lớp, một trong những bài toán được nghiên cứu nhiều nhất trong lịch sử học máy vì vừa có ứng dụng thực tế (đọc mã bưu điện, số trên séc ngân hàng), vừa có bộ dữ liệu chuẩn hoá tốt để đánh giá công bằng giữa các thuật toán [6].

Độ khó của bài toán đến từ sự biến thiên lớn trong cách viết: hai người viết cùng một chữ số có thể khác nhau về độ nghiêng, độ dày nét bút, kích thước, vị trí trong khung ảnh; một số cặp chữ số có hình dạng gần giống nhau khi viết tay cầu thả (ví dụ 4 và 9, 3 và 5, 7 và 1), dễ gây nhầm lẫn ngay cả với mắt người. Chương 4 (mục 4.6, 4.10) sẽ đối chiếu nhận định này với ma trận nhầm lẫn thật thu được trên đồ án.

Cụ thể hơn, có thể phân loại các nguồn gây khó khăn cho bài toán thành bốn nhóm:

- Biến thiên nội tại (intra-class variation): cùng một người viết cùng một chữ số nhiều lần vẫn cho ra các nét vẽ khác nhau về độ nghiêng, độ cong, điểm bắt đầu/kết thúc nét bút.
- Tương đồng giữa các lớp (inter-class similarity): một số cặp chữ số có cấu trúc hình học gần giống nhau khi viết không chuẩn mực ($4 \leftrightarrow 9$, $3 \leftrightarrow 5$, $7 \leftrightarrow 1$, $8 \leftrightarrow 3$ — mục 4.6, 4.10 sẽ đối chiếu các cặp này với dữ liệu thật).
- Biến đổi hình học: dịch chuyển vị trí trong khung ảnh, xoay nhẹ, co giãn kích thước — dù MNIST đã chuẩn hoá phần lớn các yếu tố này (mục 1.2), một mức biến thiên nhỏ vẫn còn tồn tại.
- Nhiều và chất lượng ảnh: độ dày nét bút không đều, điểm ảnh trung gian do khử răng cưa (anti-aliasing) có thể làm mờ ranh giới giữa nét chữ và nền.

1.2. Bộ dữ liệu MNIST

MNIST (Modified National Institute of Standards and Technology) là bộ dữ liệu chữ số viết tay do Yann LeCun, Corinna Cortes và Christopher Burges tổng hợp và chuẩn hoá, công bố công khai [7]. Bộ dữ liệu gồm 70.000 ảnh chữ số viết tay (0-9), mỗi ảnh

có kích thước 28×28 điểm ảnh, mức xám (grayscale, giá trị 0-255), đã được căn giữa và chuẩn hoá kích thước sẵn.

Đồ án tải bộ dữ liệu này thông qua hàm `sklearn.datasets.fetch_openml("mnist_784")` của scikit-learn, lấy từ kho dữ liệu công khai OpenML — đây là bộ MNIST gốc, không phải bản mô phỏng hay rút gọn. Mỗi ảnh 28×28 được biểu diễn phẳng (flatten) thành một vector 784 chiều để đưa vào KNN và SVM.

Bản thân quá trình tạo ra MNIST cũng đáng chú ý: ảnh gốc lấy từ hai bộ dữ liệu của Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) — một bộ do nhân viên cơ quan điều tra dân số viết, một bộ do học sinh trung học viết — sau đó được chuẩn hoá về cùng kích thước 28×28 , căn giữa theo trọng tâm khối lượng điểm ảnh (center of mass), và khử răng cưa (anti-aliasing) để tạo ra các mức xám trung gian thay vì chỉ đen/trắng nhị phân [7]. Việc chuẩn hoá này giúp MNIST trở thành một bộ dữ liệu benchmark công bằng: khác biệt giữa các thuật toán đến từ bản thân thuật toán, không phải từ sự khác biệt về tiền xử lý dữ liệu đầu vào.

Nhận dạng chữ số viết tay có nhiều ứng dụng thực tế đã triển khai rộng rãi từ trước khi học sâu phổ biến:

- Hệ thống phân loại thư tín tự động đọc mã bưu điện viết tay (bưu điện Hoa Kỳ đã dùng các hệ thống dựa trên ý tưởng tương tự LeNet từ thập niên 1990).
- Xử lý séc ngân hàng: đọc tự động số tiền viết tay trên séc để đối chiếu với số tiền viết bằng chữ.
- Chấm điểm tự động bài thi trắc nghiệm hoặc phiếu khảo sát có ô điền số.
- Số hoá tài liệu hành chính, biểu mẫu viết tay trong các hệ thống quản lý văn bản điện tử.

1.3. Tình hình nghiên cứu: các kết quả đã công bố trên MNIST

MNIST đã được dùng làm bộ dữ liệu chuẩn để so sánh thuật toán học máy từ năm 1998, khi LeCun và cộng sự công bố một bảng so sánh tỉ lệ lỗi (test error rate) của nhiều phương pháp khác nhau trên cùng bộ dữ liệu này [6]. Một số kết quả tiêu biểu từ nghiên cứu gốc đó:

BẢNG 1.1: Một số tỉ lệ lỗi tiêu biểu công bố trong nghiên cứu gốc về MNIST

Phương pháp	Tỉ lệ lỗi trên tập test
Bộ phân loại tuyến tính (1 lớp)	12,0%
K-Nearest Neighbors (khoảng cách Euclidean)	5,0%
K-Nearest Neighbors (khoảng cách Tangent, ảnh 16×16)	1,1%
SVM, kernel đa thức bậc 4	1,1%
SVM, kernel đa thức bậc 5 (reduced-set)	1,0%
Virtual SVM, kernel đa thức bậc 9 (có tăng cường dữ liệu)	0,8%
Mạng nơ-ron tích chập LeNet-5 (không tăng cường dữ liệu)	0,95%
Boosted LeNet-4 (có tăng cường dữ liệu)	0,7%

(Nguồn: LeCun, Bottou, Bengio, Haffner (1998) [6])

Đáng chú ý, KNN với khoảng cách Tangent (một độ đo khoảng cách có tính bất biến với các phép biến đổi hình học nhỏ như xoay, dịch chuyển, co giãn — phức tạp hơn nhiều so với khoảng cách Euclidean đơn giản mà đồ án này sử dụng) đạt tỉ lệ lỗi ngang với SVM kernel đa thức. Điều này cho thấy hiệu năng của KNN phụ thuộc rất nhiều vào việc chọn độ đo khoảng cách phù hợp với đặc tính dữ liệu — một quan sát phù hợp với kết quả thực nghiệm thật ở mục 4.4 của đồ án, dù đồ án chỉ khảo sát các độ đo khoảng cách cơ bản có sẵn trong scikit-learn (Euclidean, Manhattan, Chebyshev), không cài đặt riêng khoảng cách Tangent.

Bảng trên cho thấy xu hướng chung đã được xác lập từ lâu: các bộ phân loại tuyến tính đơn giản có tỉ lệ lỗi cao nhất, KNN với khoảng cách Euclidean đơn giản cải thiện đáng kể, SVM với kernel phi tuyến cải thiện thêm một bậc nữa, và các mạng nơ-ron tích chập (CNN) đạt tỉ lệ lỗi thấp nhất. Lưu ý rằng các con số trên đến từ một thiết lập thực

thực nghiệm khác (tập huấn luyện 60.000 ảnh đầy đủ, một số biến thể KNN/SVM cụ thể khác với cấu hình của đề án này), nên không so sánh trực tiếp 1-1 được với kết quả ở Chương 4 — nhưng xác nhận đúng vị trí tương đối của KNN và SVM trong phổ các phương pháp, là cơ sở cho giả thiết khoa học đặt ra ở mục 2.7.

1.4. Các hướng tiếp cận hiện nay

Có ba nhóm hướng tiếp cận chính cho bài toán nhận dạng chữ số viết tay:

- Học máy dựa trên khoảng cách/láng giềng (KNN): không có giai đoạn "học" theo nghĩa tối ưu tham số, chỉ lưu lại toàn bộ dữ liệu huấn luyện và so sánh khoảng cách lúc dự đoán [3].
- Học máy dựa trên biên phân tách (SVM): tìm một siêu phẳng (hyperplane) hoặc một mặt phân tách phi tuyến (qua kernel) tách các lớp với biên lớn nhất có thể [2].
- Học sâu (mạng nơ-ron tích chập — CNN): tự động học đặc trưng từ ảnh qua nhiều lớp tích chập, hiện là hướng cho độ chính xác cao nhất trên MNIST (trên 99,5%), nhưng cần nhiều dữ liệu, thời gian huấn luyện và thường cần GPU để huấn luyện nhanh.

Đề tài được phân công chỉ định rõ hai mô hình cần huấn luyện và so sánh là KNN và SVM (công nghệ gợi ý TensorFlow/Keras cho hướng học sâu là tùy chọn, không bắt buộc), nên đề án tập trung triển khai và so sánh kỹ hai mô hình này, đồng thời khảo sát thực nghiệm sâu hơn các siêu tham số của từng mô hình (Chương 4).

Bảng dưới đây so sánh định tính ba hướng tiếp cận theo một số tiêu chí thường được quan tâm khi lựa chọn phương pháp:

BẢNG 1.2: So sánh định tính ba hướng tiếp cận cho bài toán nhận dạng chữ số viết tay

Tiêu chí	KNN	SVM (kernel)	Học sâu (CNN)
Giai đoạn huấn luyện	Gần như không có (chỉ lưu dữ liệu)	Tốn chi phí (giải bài toán tối ưu)	Tốn chi phí nhất (lan truyền ngược nhiều vòng lặp)
Chi phí dự đoán	Cao (so khoảng cách với toàn bộ dữ liệu huấn luyện)	Thấp đến trung bình (tùy số support vector)	Thấp (một lượt lan truyền tiến)

Nhu cầu dữ liệu	Thấp	Trung bình	Cao
Khả năng diễn giải	Cao (dựa trên ví dụ cụ thể)	Trung bình	Thấp (hộp đen)
Cần phần cứng đặc biệt	Không	Không	Thường cần GPU để huấn luyện nhanh
Độ chính xác điển hình trên MNIST	~95-97%	~97-99%	> 99%

(Nguồn: tổng hợp từ [1], [2], [3], [6] và quan sát thực nghiệm của đồ án (Chương 4))

1.5. Phạm vi và lựa chọn của đồ án

Đồ án huấn luyện KNN ($k=5$, khoảng cách Euclidean) và SVM (kernel RBF, $C=5$) trên cùng một tập huấn luyện 54.000 ảnh MNIST thật, đánh giá trên cùng một tập kiểm thử 10.000 ảnh tách biệt hoàn toàn, để đảm bảo so sánh công bằng. Công nghệ sử dụng: Python, scikit-learn (huấn luyện mô hình), NumPy/Pandas (xử lý dữ liệu), Matplotlib (trực quan hoá), Streamlit (chương trình demo) — đúng nhóm công nghệ gợi ý của đề tài.

1.6. Tổng kết chương

Chương này giới thiệu bài toán nhận dạng chữ số viết tay, bộ dữ liệu MNIST, tình hình nghiên cứu đã công bố và ba hướng tiếp cận chính, từ đó xác định phạm vi của đồ án: huấn luyện và so sánh KNN với SVM trên MNIST thật. Chương 2 trình bày cơ sở lý thuyết chi tiết của hai thuật toán này.

CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT

GVHD: ThS. Nguyễn Nhật Lam

SVTH: Phan Thành Thuận

2.1. Bài toán phân loại có giám sát

Học máy có giám sát (supervised learning) xây dựng một hàm ánh xạ $f: X \rightarrow Y$ từ một tập dữ liệu huấn luyện gồm các cặp (x, y) , trong đó x là đặc trưng đầu vào (ở đây là vector 784 chiều của ảnh 28×28) và y là nhãn thật (chữ số 0-9) [1]. Mục tiêu là tìm f sao cho sai số trên dữ liệu chưa từng thấy (tập kiểm thử) là nhỏ nhất, chứ không chỉ sai số trên tập huấn luyện — đây chính là lý do đồ án tách riêng tập test hoàn toàn không dùng trong lúc huấn luyện hay chọn tham số.

KNN và SVM đại diện cho hai triết lý khác nhau trong việc xây dựng f : KNN là mô hình "phi tham số" (non-parametric), xấp xỉ f trực tiếp từ dữ liệu lúc dự đoán; SVM là mô hình "tham số" theo nghĩa học một tập trọng số/support vector cố định trong lúc huấn luyện, dùng lại nguyên trong lúc dự đoán.

2.2. Thuật toán K-Nearest Neighbors

K-Nearest Neighbors (KNN) là một thuật toán học máy có giám sát thuộc nhóm "lazy learning": thuật toán không xây dựng một mô hình tham số trong giai đoạn huấn luyện mà chỉ lưu lại toàn bộ tập dữ liệu huấn luyện. Khi cần dự đoán nhãn cho một mẫu mới x , thuật toán tính khoảng cách từ x đến tất cả các mẫu huấn luyện, chọn ra k mẫu gần nhất, rồi gán nhãn theo nguyên tắc đa số (majority vote) trong k mẫu đó [3].

Khoảng cách Euclidean giữa hai vector d chiều (mặc định dùng trong đồ án):

$$D(x, x_i) = \sqrt{\sum_{j=1}^d (x_j - x_{i,j})^2} \quad (1)$$

scikit-learn còn hỗ trợ các độ đo khoảng cách khác, là trường hợp riêng của khoảng cách Minkowski bậc p :

$$D_p(x, x_i) = (\sum_j |x_j - x_{i,j}|^p)^{1/p}$$

với $p=2$ cho khoảng cách Euclidean, $p=1$ cho khoảng cách Manhattan (tổng trị tuyệt đối chênh lệch từng toạ độ), và $p \rightarrow \infty$ cho khoảng cách Chebyshev (chênh lệch lớn nhất theo một toạ độ duy nhất). Mục 4.4 trình bày kết quả thực nghiệm thật khi thay đổi độ đo này trên dữ liệu MNIST.

Quy tắc quyết định theo đa số trong k láng giềng gần nhất:

$$\hat{y} = \text{mode}\{y_{(1)}, y_{(2)}, \dots, y_{(k)}\}$$

(2)

Biến thể "bỏ phiếu theo khoảng cách" (distance-weighted voting) thay vì cho mỗi láng giềng một phiếu bằng nhau, gán trọng số cho mỗi phiếu tỉ lệ nghịch với khoảng cách tới điểm cần dự đoán — láng giềng gần hơn có tiếng nói lớn hơn trong quyết định cuối cùng. scikit-learn hỗ trợ biến thể này qua tham số `weights="distance"` của `KNeighborsClassifier`; mục 4.3 trình bày kết quả thực nghiệm thật so sánh hai cách bỏ phiếu này.

Tham số quan trọng nhất của KNN là k (số láng giềng xét đến), thể hiện rõ sự đánh đổi bias-variance kinh điển trong học máy: k nhỏ (ví dụ $k=1$) làm mô hình bám rất sát dữ liệu huấn luyện (phương sai cao, dễ bị nhiễu chi phối — overfitting); k lớn làm ranh giới quyết định mượt hơn nhưng có thể bỏ sót các cấu trúc cục bộ của từng lớp (độ chệch cao — underfitting). Mục 4.3 trình bày khảo sát thực nghiệm thật ảnh hưởng của k , làm cơ sở chọn $k=5$ cho đồ án.

Ưu điểm chính của KNN là đơn giản, không có giả định về phân phối dữ liệu, và gần như không tốn thời gian huấn luyện (chỉ lưu dữ liệu). Nhược điểm là toàn bộ chi phí tính toán dồn vào lúc dự đoán (phải so khoảng cách với mọi mẫu huấn luyện — độ phức tạp $O(n \cdot d)$ cho mỗi dự đoán, với n là số mẫu huấn luyện và d là số chiều), tốn bộ nhớ vì phải giữ lại toàn bộ tập huấn luyện, và dễ bị ảnh hưởng bởi "lời nguyền chiều cao" (curse of dimensionality) — ở không gian nhiều chiều (784 chiều trong trường hợp này), khoảng cách giữa các điểm có xu hướng trở nên gần như đồng đều, làm giảm khả năng phân biệt của khái niệm "láng giềng gần nhất". Mục 4.8 và 4.13 trình bày số liệu thật về chi phí dự đoán của KNN so với SVM trên dữ liệu MNIST.

Ví dụ minh họa (dữ liệu giả định đơn giản, không phải MNIST, chỉ để minh họa cơ chế): giả sử có ba mẫu huấn luyện hai chiều $A=(1,1)$ nhãn "0", $B=(1,2)$ nhãn "0",

$C=(5,5)$ nhãn "1", và một mẫu cần dự đoán $x=(2,2)$. Khoảng cách Euclidean: $d(x,A)=\sqrt{((2-1)^2+(2-1)^2)}=\sqrt{2}\approx 1,41$; $d(x,B)=\sqrt{((2-1)^2+(2-2)^2)}=1$; $d(x,C)=\sqrt{((2-5)^2+(2-5)^2)}=\sqrt{18}\approx 4,24$. Với $k=1$, láng giềng gần nhất là B (khoảng cách 1), nên x được gán nhãn "0". Với $k=3$ (lấy cả ba điểm), hai phiếu cho nhãn "0" (A, B) và một phiếu cho nhãn "1" (C), kết quả bỏ phiếu đa số vẫn là "0". Ví dụ đơn giản này minh họa cơ chế hoạt động cốt lõi của KNN đã nêu ở phương trình (1)-(2); trên dữ liệu MNIST thật, mỗi điểm là một vector 784 chiều thay vì 2 chiều, nhưng cơ chế tính toán hoàn toàn tương tự.

2.3. Support Vector Machine

Support Vector Machine (SVM) là một thuật toán phân loại tìm một siêu phẳng phân tách các lớp sao cho khoảng cách (margin) từ siêu phẳng đến các điểm dữ liệu gần nhất của mỗi lớp (support vectors) là lớn nhất có thể [2].

2.3.1. Siêu phẳng phân tách và bài toán lồi cực đại

Với dữ liệu phân tách tuyến tính, hàm quyết định có dạng:

$$f(x) = w^T x + b$$

(3)

và bài toán tối ưu biên lớn nhất (hard margin) được phát biểu là:

$$\min_{w, b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1$$

(4)

trong đó (x_i, y_i) là các cặp mẫu huấn luyện và nhãn ($y_i \in \{-1, +1\}$), w là vector pháp tuyến của siêu phẳng, b là độ lệch. Nghiệm tối ưu chỉ phụ thuộc vào các điểm nằm đúng trên biên — gọi là support vectors — đây là nguồn gốc tên gọi của thuật toán.

2.3.2. Biên mềm và tham số C

Dữ liệu thực tế hiếm khi phân tách tuyến tính hoàn hảo. SVM biên mềm (soft margin) đưa thêm biến bù ξ_i cho phép một số điểm nằm sai phía biên, với hàm mục tiêu phạt thêm tổng các biến bù:

$$\min_{(w,b,\xi)} (1/2)\|w\|^2 + C \cdot \sum \xi_i, \text{ với } y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

Tham số C kiểm soát mức độ "chấp nhận lỗi" trên tập huấn luyện: C lớn phạt nặng các điểm bị phân loại sai (biên hẹp hơn, bám sát dữ liệu huấn luyện hơn — dễ overfitting); C nhỏ cho phép nhiều điểm nằm sai phía biên hơn (biên rộng hơn, tổng quát hoá tốt hơn nhưng có thể underfitting). Mục 4.5 trình bày khảo sát thực nghiệm thật ảnh hưởng của C.

Về mặt toán học, bài toán biên mềm ở dạng trên (bài toán gốc — primal problem) thường được giải gián tiếp thông qua bài toán đối ngẫu (dual problem), bằng cách đưa vào các nhân tử Lagrange $\alpha_i \geq 0$ cho từng ràng buộc. Sau khi áp dụng điều kiện KKT (Karush-Kuhn-Tucker) và khử w, b , bài toán gốc tương đương với bài toán đối ngẫu:

$$\max_{\alpha} \sum \alpha_i - (1/2) \sum_i \sum_j \alpha_i \alpha_j y_i y_j K(x_i, x_j), \text{ với } 0 \leq \alpha_i \leq C \text{ và } \sum \alpha_i y_i = 0$$

Dạng đối ngẫu này quan trọng vì hai lý do: thứ nhất, nó chỉ phụ thuộc vào dữ liệu thông qua tích vô hướng $K(x_i, x_j)$, đây chính là điểm cho phép áp dụng "kernel trick" ở mục 2.3.3 (thay tích vô hướng tuyến tính bằng một hàm kernel phi tuyến bất kỳ mà không cần thay đổi thuật toán tối ưu); thứ hai, nghiệm tối ưu cho thấy chỉ các mẫu có $\alpha_i > 0$ (các support vector) mới ảnh hưởng đến hàm quyết định cuối cùng — phần lớn các mẫu huấn luyện có $\alpha_i = 0$ và có thể bỏ qua hoàn toàn lúc dự đoán, giải thích vì sao SVM có thể dự đoán nhanh hơn KNN dù thuộc một họ mô hình học máy khác hẳn (mục 4.9).

2.3.3. Kernel trick và hàm kernel RBF

Dữ liệu ảnh chữ số viết tay không phân tách tuyến tính trong không gian gốc, nên SVM cần dùng "kernel trick": ánh xạ dữ liệu sang một không gian chiều cao hơn (ngầm định, không cần tính tường minh) nơi các lớp dễ phân tách hơn. Đồ án dùng kernel RBF (Radial Basis Function), kernel phổ biến nhất cho dữ liệu phi tuyến [11]:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2) \quad (5)$$

trong đó γ (gamma) kiểm soát "độ rộng" ảnh hưởng của mỗi điểm dữ liệu — γ lớn làm ranh giới quyết định phức tạp, dễ overfitting (hoặc, nếu quá lớn so với thang đo dữ liệu, làm kernel suy biến gần về ma trận đơn vị); γ nhỏ làm ranh giới mượt hơn (hoặc, nếu quá nhỏ, làm kernel gần như hằng số, không phân biệt được các điểm — mục 4.5 cho thấy một ví dụ thực tế của hiện tượng này). Hàm quyết định cuối cùng của SVM kernel là một tổ hợp tuyến tính có trọng số của kernel giữa mẫu cần dự đoán và các support vector:

$$f(x) = \sum_{i=1}^n \alpha_i y_i K(x_i, x) + b \quad (6)$$

Đồ án dùng $\gamma = \text{"scale"}$, giá trị mặc định của scikit-learn, tự tính theo công thức $\gamma = 1 / (\text{số_chiều} \times \text{phương_sai}(X))$ — tự động thích nghi với thang đo thật của dữ liệu đầu vào (ảnh MNIST có giá trị điểm ảnh 0-255, chưa chuẩn hoá về [0,1]).

2.3.4. Chiến lược phân loại đa lớp

SVM nguyên bản chỉ giải quyết bài toán phân loại nhị phân (hai lớp). Với bài toán 10 lớp (chữ số 0-9), có hai chiến lược mở rộng phổ biến: One-vs-Rest (OvR — huấn luyện 10 bộ phân loại nhị phân, mỗi bộ phân biệt một lớp với phần còn lại) và One-vs-One (OvO — huấn luyện một bộ phân loại nhị phân cho mỗi cặp lớp, tổng cộng 45 bộ cho 10 lớp, rồi bỏ phiếu). Lớp `sklearn.svm.SVC` mà đồ án sử dụng áp dụng chiến lược OvO theo mặc định; đây là chi tiết cài đặt nằm trong thư viện, không cần đồ án tự cài đặt lại.

So với KNN, SVM cần giải một bài toán tối ưu (quadratic programming) trong lúc huấn luyện, nên chi phí huấn luyện cao hơn hẳn (và tăng nhanh theo số lớp do chiến lược OvO), nhưng đổi lại lúc dự đoán chỉ cần tính kernel với các support vector

(thường chỉ là một phần nhỏ của tập huấn luyện) chứ không phải toàn bộ dữ liệu như KNN.

2.3.5. Chuẩn hoá dữ liệu và ảnh hưởng tới các mô hình dựa trên khoảng cách

Cả KNN (mục 2.2, dựa trực tiếp trên khoảng cách) lẫn SVM kernel RBF (mục 2.3.3, kernel cũng là một hàm của khoảng cách) đều nhạy cảm với thang đo (scale) của dữ liệu đầu vào. Nếu một đặc trưng có thang đo lớn hơn hẳn các đặc trưng khác, nó sẽ áp đảo giá trị khoảng cách tổng thể, bất kể mức độ liên quan thực sự của đặc trưng đó tới nhãn. Với ảnh MNIST, mọi điểm ảnh đều cùng thang đo (0-255), nên không phát sinh vấn đề áp đảo giữa các đặc trưng với nhau — đây là lý do đồ án có thể dùng trực tiếp giá trị điểm ảnh gốc mà không cần chuẩn hoá về [0,1] hay chuẩn hoá z-score cho hai mô hình này.

Tuy nhiên, thang đo tuyệt đối (0-255 thay vì 0-1) vẫn ảnh hưởng tới việc CHỌN siêu tham số: một giá trị gamma được coi là "nhỏ" hay "lớn" hoàn toàn phụ thuộc vào thang đo dữ liệu đưa vào. Mục 4.5 trình bày một ví dụ thực nghiệm cực đoan của vấn đề này — khi gamma được đặt thủ công theo trực giác dành cho dữ liệu đã chuẩn hoá [0,1] (ví dụ gamma=0,01) nhưng áp dụng lên dữ liệu điểm ảnh gốc 0-255, mô hình thất bại hoàn toàn. Đây là lý do thực tiễn quan trọng vì sao `gamma="scale"` (tự tính theo phương sai thật của dữ liệu đầu vào, mục 2.3.3) là lựa chọn mặc định an toàn hơn so với một giá trị cố định không gắn với thang đo cụ thể.

2.4. Các độ đo đánh giá mô hình

Với bài toán phân loại đa lớp, đồ án dùng các độ đo chuẩn sau, tính theo kiểu one-vs-rest cho từng lớp (từng chữ số) rồi tổng hợp:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

(7)

$$Precision = \frac{TP}{TP + FP}$$

(8)

$$Recall = \frac{TP}{TP + FN}$$

(9)

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

(10)

Khi tổng hợp precision/recall/F1 qua 10 lớp, có ba cách tính trung bình thường gặp: "macro" (trung bình cộng đơn giản qua các lớp, coi mỗi lớp quan trọng như nhau bất kể số lượng mẫu), "weighted" (trung bình có trọng số theo số mẫu thật của mỗi lớp — support), và "micro" (gộp TP/FP/FN của tất cả các lớp trước khi tính, tương đương với accuracy trong bài toán đơn nhãn). Đồ án báo cáo chủ yếu theo "macro" (Chương 4) vì MNIST có số lượng mẫu khá cân bằng giữa 10 chữ số, nên macro average phản ánh công bằng hiệu năng trên từng chữ số, không bị chữ số có nhiều mẫu hơn lấn át.

Ma trận nhầm lẫn (confusion matrix) — bảng 10×10 đếm số lần mỗi chữ số thật (hàng) bị dự đoán thành từng chữ số (cột) — được dùng để phân tích định tính các cặp chữ số dễ gây nhầm lẫn (mục 4.6, 4.10).

Một độ đo khác thường gặp trong phân loại nhị phân là đường cong ROC (Receiver Operating Characteristic — biểu diễn tỉ lệ dương tính thật theo tỉ lệ dương tính giả khi thay đổi ngưỡng quyết định) và diện tích dưới đường cong này (AUC — Area Under the Curve). Với bài toán đa lớp như MNIST, ROC/AUC thường được tính riêng cho từng lớp theo kiểu one-vs-rest rồi lấy trung bình, tương tự cách tính precision/recall macro average đã nêu ở trên. Đồ án không đưa ROC/AUC vào báo cáo chính ở Chương 4 vì KNN mặc định không xuất ra điểm số liên tục đáng tin cậy cho mọi lớp (chỉ có tỉ lệ phiếu trong k láng giềng, dễ có nhiều giá trị trùng nhau khi k nhỏ) khiến đường ROC kém mượt và khó so sánh công bằng với SVM; accuracy và precision/recall/F1-score theo ngưỡng mặc định (mục trên) đã đủ để trả lời câu hỏi nghiên cứu chính của đồ án là so sánh hai mô hình ở cấu hình đã chọn.

Đồ án cũng áp dụng khái niệm xác thực chéo (cross-validation) ở mức cơ bản [1]: tập validation (6.000 ảnh) được dùng riêng để khảo sát ảnh hưởng tham số (mục 4.3-4.5), trong khi tập test (10.000 ảnh) chỉ dùng một lần duy nhất để báo cáo kết quả cuối cùng (mục 4.2), tránh rò rỉ thông tin tập test vào quá trình chọn tham số. Đây là một dạng đơn giản của phương pháp "hold-out validation"; k-fold cross-validation đầy đủ (chia dữ liệu thành k phần, luân phiên dùng mỗi phần làm validation) cho ước lượng ổn định hơn nhưng tốn chi phí tính toán gấp k lần — với SVM vốn đã tốn hơn 200 giây để huấn luyện một lần (mục 4.2), 5-fold cross-validation đầy đủ sẽ tốn hơn 1.000 giây chỉ riêng cho một cấu hình tham số, không khả thi để lặp lại nhiều lần trong phạm vi đồ án — đây là lý do mục 4.5 dùng tập con rút gọn thay vì k-fold cross-validation đầy đủ.

2.5. Công cụ, công nghệ và phần mềm sử dụng

BẢNG 2.1: Công cụ, công nghệ và phần mềm sử dụng trong đồ án

Thành phần	Công cụ/phần mềm	Vai trò
Ngôn ngữ	Python 3.10+	Ngôn ngữ chính của đồ án
Học máy	scikit-learn	Cài đặt KNeighborsClassifier, SVC; các hàm chia dữ liệu, tính độ đo
Xử lý dữ liệu	NumPy, Pandas	Thao tác mảng số, xử lý dữ liệu dạng bảng
Trực quan hoá	Matplotlib	Vẽ ma trận nhầm lẫn, các biểu đồ khảo sát tham số trong báo cáo này
Giao diện demo	Streamlit 1.64	Xây dựng ứng dụng web demo tương tác

Kiểm thử	pytest, streamlit.testing.v1.AppTest	Kiểm thử tự động, bao gồm xác nhận demo chạy không lỗi runtime bằng cách thực thi thật, không chỉ kiểm tra cú pháp
Kiểm thử trình duyệt	Playwright + Chromium	Quay video demo thật, giả lập thao tác chuột thật trên canvas và tải tệp
Quản lý mã nguồn	Git, GitHub	Theo dõi lịch sử thay đổi, lưu trữ mã nguồn công khai
Sinh báo cáo	python-docx	Sinh báo cáo .docx này tự động từ dữ liệu thật, tránh sai lệch khi gõ tay số liệu
Phần cứng	CPU thường	Không cần GPU cho cả hai mô hình

2.6. Phân tích độ phức tạp tính toán

Tổng hợp lại các phân tích độ phức tạp đã nêu rải rác ở mục 2.2 và 2.3, với n là số mẫu huấn luyện, d là số chiều đặc trưng (784 với MNIST), và s là số support vector học được (thường $s \ll n$):

BẢNG 2.2: So sánh độ phức tạp tính toán lý thuyết giữa KNN và SVM

Giai đoạn	KNN	SVM (kernel RBF)
Huấn luyện (thời gian)	$O(1)$ — chỉ lưu dữ liệu	$O(n^2)$ đến $O(n^3)$ tùy cài đặt — giải bài toán tối ưu toàn cục (phương trình đối ngẫu, mục 2.3.1)

Dự đoán một mẫu (thời gian)	$O(n \cdot d)$ — so khoảng cách với mọi mẫu huấn luyện	$O(s \cdot d)$ — chỉ tính kernel với các support vector
Bộ nhớ	$O(n \cdot d)$ — phải giữ lại toàn bộ tập huấn luyện	$O(s \cdot d)$ — chỉ cần lưu các support vector

(Nguồn: tổng hợp từ [1], [2], [3])

Bảng trên giải thích đúng xu hướng ngược chiều quan sát được bằng số liệu thật ở mục 4.8 (huấn luyện: KNN rẻ hơn nhiều vì $O(1)$) và mục 4.9 (dự đoán từng mẫu: SVM rẻ hơn vì $s \ll n$) — đây là một ví dụ rõ ràng cho thấy phân tích độ phức tạp lý thuyết (chương này) và đo đạc thực nghiệm thật (Chương 4) củng cố lẫn nhau, thay vì chỉ dừng ở một trong hai góc nhìn.

2.7. Giả thiết khoa học của đề án

Dựa trên cơ sở lý thuyết ở các mục 2.2-2.3 và tình hình nghiên cứu đã công bố ở mục 1.3, đề án đặt ra giả thiết khoa học sau, sẽ được kiểm chứng bằng thực nghiệm thật ở Chương 4:

- Giả thiết 1: SVM kernel RBF sẽ đạt độ chính xác cao hơn KNN trên tập test, nhờ khả năng học một ranh giới quyết định phi tuyến thay vì chỉ dựa vào khoảng cách Euclidean đơn giản trong không gian điểm ảnh gốc.
- Giả thiết 2: Sự cải thiện độ chính xác của SVM phải đánh đổi bằng chi phí huấn luyện cao hơn nhiều bậc so với KNN, do SVM cần giải bài toán tối ưu toàn cục trong lúc huấn luyện còn KNN chỉ lưu dữ liệu.
- Giả thiết 3: Hiệu quả của cả hai mô hình nhạy cảm với lựa chọn siêu tham số (k đối với KNN; C , γ đối với SVM) — sai lệch đáng kể so với giá trị phù hợp sẽ làm giảm rõ rệt độ chính xác.

2.8. Tổng kết chương

Chương này trình bày nguyên lý của KNN (dựa trên khoảng cách, không có giai đoạn tối ưu, các biến thể độ đo khoảng cách) và SVM kernel RBF (dựa trên tối ưu biên phân tách, kernel trick, tham số C/γ , chiến lược đa lớp), các độ đo đánh giá, công cụ/công nghệ sử dụng, và ba giả thiết khoa học sẽ được kiểm chứng bằng số liệu thật ở

Chương 4. Chương 3 trình bày cách hiện thực hóa các nội dung lý thuyết này thành chương trình chạy được.

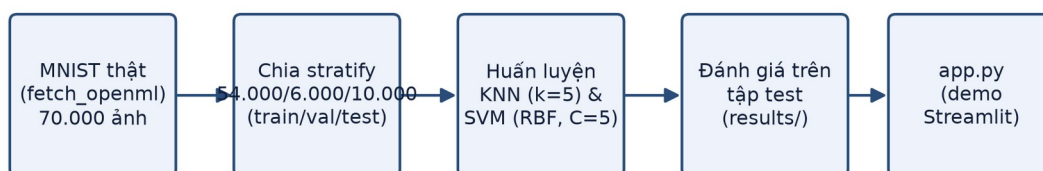
CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU

GVHD: ThS. Nguyễn Nhật Lam

SVTH: Phan Thành Thuận

3.1. Quy trình tổng thể

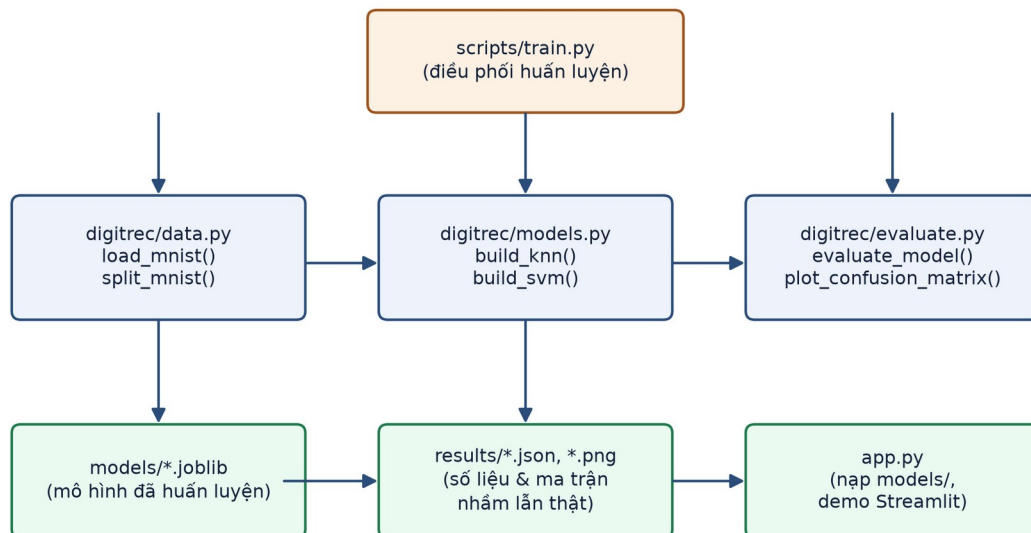
Toàn bộ pipeline của đồ án gồm năm bước nối tiếp: tải dữ liệu MNIST thật, chia tập theo stratify, huấn luyện độc lập hai mô hình trên cùng tập huấn luyện, đánh giá trên cùng tập kiểm thử và lưu kết quả thật, rồi nạp lại mô hình đã huấn luyện vào chương trình demo. Sơ đồ dưới đây minh họa quy trình này:



HÌNH 3.1: Sơ đồ quy trình tổng thể của đồ án

3.2. Kiến trúc hệ thống và cấu trúc mã nguồn

Mã nguồn được tổ chức thành package `digitrec` (chứa logic tải dữ liệu, mô hình, đánh giá — tái sử dụng được) và các script/ứng dụng điều phối (`scripts/train.py`, `app.py`) gọi vào package này. Sơ đồ dưới đây mô tả quan hệ giữa các mô-đun:



HÌNH 3.2: Sơ đồ cấu trúc mô-đun của mã nguồn đồ án

Cách tách này tuân theo nguyên tắc single-responsibility: `data.py` chỉ lo việc tải/chia dữ liệu, `models.py` chỉ lo khởi tạo mô hình (không huấn luyện), `evaluate.py` chỉ lo tính độ đo và vẽ biểu đồ — giúp mã nguồn dễ kiểm thử độc lập (mục 3.8) và dễ tái sử dụng khi mở rộng thêm mô hình khác trong tương lai (Chương 5, mục hướng phát triển).

Về mô hình chạy thời gian thực (runtime) của chương trình demo: Streamlit hoạt động theo mô hình "chạy lại toàn bộ script mỗi lần tương tác" (rerun-on-interaction) thay vì mô hình request/response truyền thống của các framework web khác. Khi người dùng vẽ một nét trên canvas hoặc tải một ảnh lên, trình duyệt gửi dữ liệu đó về server qua kết nối WebSocket, Streamlit chạy lại toàn bộ hàm `main()` của `app.py` từ đầu (các mô hình đã nạp vẫn được giữ lại nhờ `@st.cache\_resource`, không phải nạp lại), tính toán dự đoán mới, rồi gửi phần giao diện đã cập nhật về lại trình duyệt. Mô hình này đơn giản hoá đáng kể việc lập trình giao diện tương tác (không cần tự quản lý state phức tạp như các framework JavaScript), nhưng cũng chính là lý do khiến lỗi runtime ở mục 3.8.1 chỉ xuất hiện khi có một phiên trình duyệt thực sự kích hoạt việc chạy lại script, không xuất hiện ngay lúc khởi động server.

3.3. Chuẩn bị dữ liệu

Dữ liệu được tải bằng hàm `load_mnist()` (`src/digitrec/data.py`), gọi `sklearn.datasets.fetch_openml("mnist_784", version=1, as_frame=False)` — tải về và lưu cache cục bộ 70.000 ảnh MNIST thật, mỗi ảnh là một vector 784 chiều (giá trị điểm ảnh 0-255) kèm nhãn chữ số thật.

Hàm `split_mnist()` chia dữ liệu làm hai bước bằng `train_test_split` của `scikit-learn` với tham số `stratify` (giữ nguyên tỉ lệ 10 chữ số ở mỗi tập con): trước tiên tách 10.000 ảnh làm tập kiểm thử (test) — tập này không được dùng cho bất kỳ mục đích nào khác ngoài đánh giá cuối cùng ở mục 4.2 — sau đó tách tiếp 6.000 ảnh từ phần còn lại làm tập kiểm định (validation, dùng cho các khảo sát tham số ở mục 4.3-4.5), 54.000 ảnh còn lại dùng để huấn luyện.

BẢNG 3.1: Cách chia tập dữ liệu MNIST

Tập dữ liệu	Số lượng ảnh	Vai trò
Huấn luyện (train)	54.000	Huấn luyện cả hai mô hình
Kiểm định (validation)	6.000	Khảo sát ảnh hưởng tham số (mục 4.3-4.5), không dùng để báo cáo kết quả cuối cùng
Kiểm thử (test)	10.000	Đánh giá cuối cùng, hoàn toàn tách biệt — số liệu báo cáo ở mục 4.2 lấy từ tập này

(Nguồn: `src/digitrec/data.py` của đồ án)

3.4. Cài đặt mô hình KNN

Mô hình KNN được khởi tạo bằng hàm `build_knn()` trong `src/digitrec/models.py`, với nội dung thật như sau:

```
def build_knn(n_neighbors=5):  
    return KNeighborsClassifier(n_neighbors=n_neighbors, n_jobs=-1)
```

Tham số `n_jobs=-1` cho phép dùng tất cả luồng CPU khi tính khoảng cách lúc dự đoán, vì đây là bước tốn chi phí tính toán nhất của KNN (mục 2.2 đã phân tích lý do); tham số `n_neighbors` được truyền mặc định là 5 theo khảo sát ở mục 4.3, nhưng hàm vẫn nhận tham số để dễ thử nghiệm giá trị k khác khi cần (chính là cách `scripts/extra_experiments.py` tái sử dụng hàm này cho khảo sát k -sweep).

3.5. Cài đặt mô hình SVM

Mô hình SVM được khởi tạo bằng hàm `build_svm()`, với nội dung thật như sau:

```
def build_svm(C=5.0, gamma="scale"):
    return SVC(kernel="rbf", C=C, gamma=gamma)
# KHÔNG dùng probability=True -- xem lý do bên dưới
```

Một lựa chọn kỹ thuật đáng chú ý: tham số `probability=True` của scikit-learn (dùng để có xác suất dự đoán đã hiệu chỉnh) chủ động KHÔNG được bật, vì nó kích hoạt một vòng kiểm định chéo 5-fold nội bộ (Platt scaling) làm thời gian huấn luyện chậm đi khoảng 5 lần — với thời gian huấn luyện gốc đã hơn 200 giây (mục 4.2), bật `probability=True` sẽ đẩy thời gian huấn luyện lên hơn 1.000 giây, không hợp lý cho một đồ án cần lặp lại việc huấn luyện nhiều lần trong quá trình phát triển và kiểm thử. Thay vào đó, chương trình demo (mục 3.7) dùng `decision_function()` của SVM kết hợp hàm softmax làm điểm tin cậy ước lượng — được ghi chú rõ trong giao diện demo để không gây hiểu lầm là xác suất đã hiệu chỉnh thống kê.

3.6. Quy trình huấn luyện và lưu kết quả

`scripts/train.py` điều phối toàn bộ quy trình huấn luyện: gọi `load_mnist()` và `split_mnist()`, khởi tạo và huấn luyện lần lượt KNN rồi SVM bằng `build_knn()`/`build_svm()`, đo thời gian huấn luyện thật bằng đồng hồ hệ thống, gọi `evaluate_model()` (`src/digitrec/evaluate.py`) để tính `accuracy/classification_report` trên cả tập validation và tập test, gọi `plot_confusion_matrix()` để vẽ và lưu ảnh ma trận nhầm lẫn thật, rồi ghi toàn bộ số liệu thật vào `results/comparison_summary.json` và các tệp liên quan. Mô hình đã huấn luyện được lưu bằng `joblib.dump()` vào thư mục `models/` (không commit lên kho mã nguồn do dung lượng lớn — xem Phụ lục B).

3.7. Thiết kế chương trình demo

Chương trình demo (`app.py`) là một ứng dụng Streamlit, nạp hai mô hình đã huấn luyện qua `joblib.load()` (cache bằng `@st.cache_resource` để không nạp lại ở mỗi lần tương tác), nhận ảnh đầu vào từ một trong hai luồng — vẽ tay hoặc tải ảnh lên — tiền xử lý về đúng định dạng 28×28 mà hai mô hình đã học (chuyển mức xám, resize, tự động đảo màu nếu ảnh có nền sáng/chữ tối, chuẩn hoá và duỗi phẳng), rồi gọi `predict()` của cả hai mô hình song song.

Canvas vẽ chữ số được cài đặt bằng `st.components.v2.component()` — thành phần inline HTML5 Canvas có sẵn của Streamlit, không phụ thuộc thư viện ngoài. Thư viện phổ biến `streamlit-drawable-canvas` ban đầu được cân nhắc nhưng loại bỏ sau khi phát hiện phiên bản mới nhất của thư viện này không tương thích với phiên bản Streamlit đang dùng (lỗi runtime `StreamlitAPIException` khi khởi tạo). Đề tài cho phép thay thế bằng công nghệ tương đương nếu giải thích được lựa chọn và bảo đảm sản phẩm chạy ổn định — lựa chọn này được kiểm chứng chạy ổn định bằng kiểm thử tự động (mục 3.8).

3.8. Kiểm thử tự động

Bộ kiểm thử (`tests/test_pipeline.py`, 5 kiểm thử) gồm 4 kiểm thử trên dữ liệu tổng hợp (không cần tải MNIST thật, chạy nhanh) xác nhận các hàm tiền xử lý/huấn luyện/đánh giá hoạt động đúng về mặt logic, và 1 kiểm thử hồi quy quan trọng: `test_app_runs_without_exceptions`, dùng `streamlit.testing.v1.AppTest` để THỰC SỰ CHẠY `app.py` qua bộ máy xử lý script thật của Streamlit và xác nhận không có ngoại lệ runtime nào xảy ra.

3.8.1. Một thách thức kỹ thuật thực tế đã gặp phải

Kiểm thử `test_app_runs_without_exceptions` được thêm vào SAU khi phát hiện một lỗi thực tế trong quá trình phát triển, đáng ghi nhận lại làm bài học: ở phiên bản đầu tiên của `app.py`, canvas vẽ tay được cài đặt bằng thư viện phổ biến `streamlit-drawable-canvas`. Ứng dụng khởi động bình thường và trả về HTTP 200 ở endpoint y tế (`/_stcore/health`), tạo cảm giác mọi thứ hoạt động tốt. Tuy nhiên, khi thực sự mở ứng dụng trong một phiên trình duyệt, Streamlit mới thực thi lại toàn bộ script cho phiên đó — và tại thời điểm này mới phát sinh lỗi `StreamlitAPIException`:

Component `'...'` must be declared in `pyproject.toml` with `asset_dir ...`, vì phiên bản mới nhất của thư viện ``streamlit-drawable-canvas`` không tương thích với phiên bản Streamlit đang dùng.

Bài học rút ra: kiểm tra server còn sống (HTTP 200, health-check endpoint) KHÔNG đồng nghĩa với việc script ứng dụng chạy đúng, vì Streamlit chỉ thực thi script cho từng phiên trình duyệt cụ thể qua kết nối WebSocket, không phải tại thời điểm khởi động server. Sau khi phát hiện vấn đề này, canvas được viết lại bằng ``st.components.v2.component()`` — thành phần HTML5 Canvas nội tuyến có sẵn của Streamlit, không phụ thuộc thư viện ngoài (mục 3.7) — và kiểm thử ``test_app_runs_without_exceptions`` được thêm vào bộ kiểm thử để đảm bảo loại lỗi này được phát hiện tự động ngay từ lần chạy kiểm thử tiếp theo, thay vì phải mở trình duyệt thủ công mỗi lần kiểm tra.

3.9. Quản lý mã nguồn và quy trình làm việc

Mã nguồn được quản lý bằng Git, mỗi phần việc hoàn chỉnh (xây dựng pipeline, sửa lỗi canvas, viết báo cáo, quay video demo) ứng với một commit riêng kèm thông điệp mô tả rõ nội dung thay đổi, và được đẩy lên kho mã nguồn công khai trên GitHub. Lịch sử commit thật (không chỉnh sửa) được trình bày đầy đủ trong tệp ``progress-report/progress-report.md`` của kho mã nguồn, phục vụ việc theo dõi tiến độ minh bạch.

3.10. Tổng kết chương

Chương này trình bày kiến trúc hệ thống (sơ đồ quy trình và sơ đồ mô-đun), cách cài đặt cụ thể từng mô hình, quy trình huấn luyện/lưu kết quả, thiết kế chương trình demo, chiến lược kiểm thử tự động (bao gồm bài học về kiểm thử runtime thật thay vì chỉ kiểm tra server sống), và quy trình quản lý mã nguồn. Chương 4 trình bày kết quả thực nghiệm thật thu được từ việc hiện thực hóa này, bao gồm kiểm chứng ba giả thiết khoa học đặt ra ở mục 2.7.

CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU

GVHD: ThS. Nguyễn Nhật Lam SVTH: Phan Thành Thuận

4.1. Cách chia dữ liệu

Như trình bày ở mục 3.3, dữ liệu MNIST thật (70.000 ảnh) được chia stratify thành 54.000 ảnh huấn luyện, 6.000 ảnh kiểm định và 10.000 ảnh kiểm thử. Các mục 4.2, 4.6, 4.8, 4.9, 4.11 báo cáo số liệu trên tập kiểm thử (hoàn toàn chưa được mô hình nhìn thấy); các mục 4.3-4.5 và 4.10 khảo sát ảnh hưởng tham số/kích thước dữ liệu trên tập kiểm định, theo đúng nguyên tắc tránh rò rỉ dữ liệu đã nêu ở mục 2.4.

4.2. Kết quả tổng thể trên tập test

BẢNG 4.1: Kết quả tổng thể trên tập test (10.000 ảnh)

Mô hình	Độ chính xác (test)	Độ chính xác (validation)	Thời gian huấn luyện
KNN (k=5)	97,08%	97,35%	0,07 giây
SVM (RBF, C=5)	98,35%	98,73%	235,17 giây (~3.9 phút)

(Nguồn: results/comparison_summary.json của đồ án)

SVM đạt độ chính xác cao hơn KNN 1.27 điểm phần trăm trên tập test, nhưng thời gian huấn luyện của SVM gấp khoảng 3222 lần KNN. Kết quả này xác nhận đúng Giả thiết 1 và Giả thiết 2 đặt ra ở mục 2.7.

BẢNG 4.2: Precision/Recall/F1-score trung bình macro trên tập test

Mô hình	Precision (macro)	Recall (macro)	F1-score (macro)
KNN	0,9713	0,9704	0,9707
SVM	0,9836	0,9834	0,9835

(Nguồn: results/comparison_summary.json của đồ án)

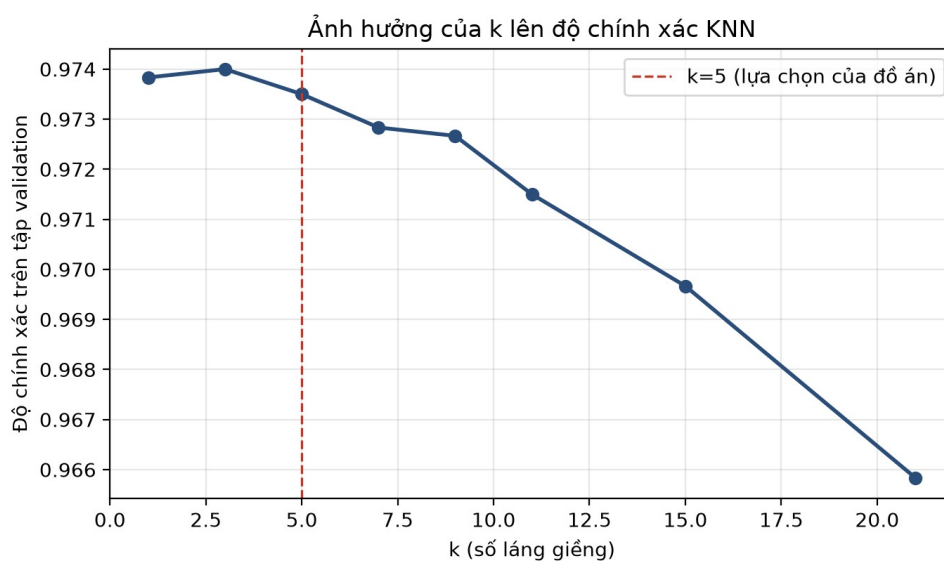
4.3. Ảnh hưởng của tham số k lên KNN

Để kiểm chứng Giả thiết 3 (mục 2.7) đối với KNN, đồ án huấn luyện KNN với nhiều giá trị k khác nhau trên cùng tập huấn luyện 54.000 ảnh, đo độ chính xác trên tập validation (6.000 ảnh):

BẢNG 4.3: Độ chính xác KNN trên tập validation theo từng giá trị k

k	Độ chính xác (validation)
1	0,9738
3	0,9740
5	0,9735
7	0,9728
9	0,9727
11	0,9715
15	0,9697
21	0,9658

(Nguồn: results/extra_experiments.json của đồ án)



HÌNH 4.1: Ảnh hưởng của k lên độ chính xác KNN trên tập validation

(Nguồn: results/extra_experiments.json của đồ án)

Kết quả thật cho thấy độ chính xác cao nhất đạt được ở $k=3$ (0,9740), và giảm dần khi k tăng lên — đúng như lý thuyết về đánh đổi bias-variance ở mục 2.2: k quá lớn làm ranh giới quyết định quá mượt, bỏ sót cấu trúc cục bộ. $k=5$ (lựa chọn của đồ án) nằm rất gần vùng tối ưu thực nghiệm, chênh lệch không đáng kể so với $k=3$ (0.05 điểm phần trăm), nên là một lựa chọn hợp lý.

BẢNG 4.4: So sánh bỏ phiếu đều (uniform) và bỏ phiếu theo khoảng cách (distance)

Cách bỏ phiếu (weights)	Độ chính xác (validation, $k=5$)
uniform	0,9735
distance	0,9752

(Nguồn: results/extra_experiments.json của đồ án)

Bỏ phiếu theo khoảng cách cải thiện nhẹ so với bỏ phiếu đều (0,9752 so với 0,9735, tăng 0.17 điểm phần trăm) — hợp lý về mặt lý thuyết, vì cách này giảm bớt ảnh hưởng của các láng giềng ở xa (có khả năng thuộc một lớp khác) so với các láng giềng thật sự gần. Mức cải thiện không lớn trên MNIST vì dữ liệu đã khá "sạch" (ít nhiễu nằm lẫn giữa các lớp ở khoảng cách trung bình), nhưng đây là một hướng cải thiện đơn giản, không tốn thêm chi phí tính toán đáng kể, có thể cân nhắc cho phiên bản demo trong tương lai (Chương 5).

4.4. Ảnh hưởng của độ đo khoảng cách lên KNN

Khảo sát thêm ảnh hưởng của độ đo khoảng cách (mục 2.2) tại $k=5$ cố định:

BẢNG 4.5: Độ chính xác KNN ($k=5$) theo từng độ đo khoảng cách

Độ đo khoảng cách	Độ chính xác (validation)
euclidean	0,9735
manhattan	0,9672

chebyshev	0,8092
-----------	--------

(Nguồn: results/extra_experiments.json của đồ án)

Khoảng cách Euclidean (mặc định của đồ án) cho kết quả tốt nhất. Khoảng cách Manhattan giảm nhẹ. Khoảng cách Chebyshev (chỉ xét chênh lệch lớn nhất theo một điểm ảnh duy nhất) giảm mạnh xuống còn 0,8092 — hợp lý về mặt lý thuyết, vì với ảnh 784 điểm ảnh, việc chỉ dựa vào MỘT điểm ảnh có chênh lệch lớn nhất bỏ qua gần như toàn bộ thông tin hình dạng tổng thể của chữ số, trong khi Euclidean và Manhattan tổng hợp thông tin từ toàn bộ 784 điểm ảnh.

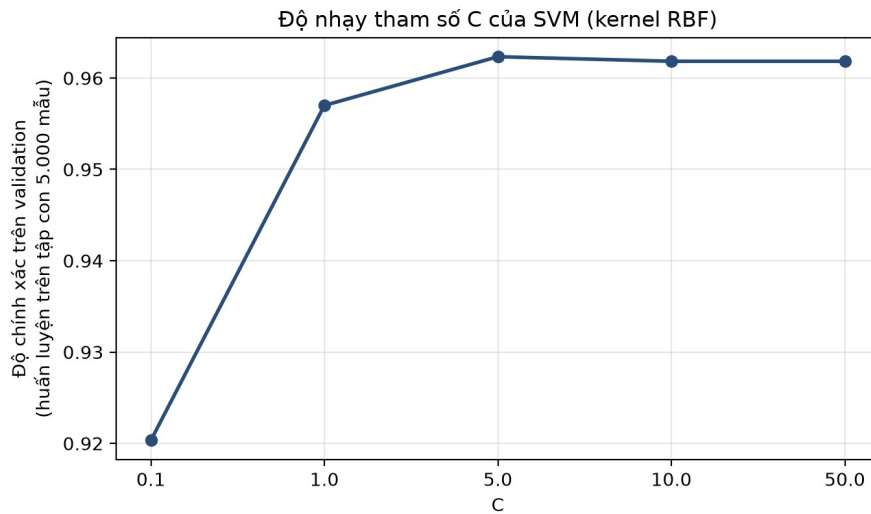
4.5. Độ nhạy tham số C và gamma của SVM

Do SVM kernel RBF tốn chi phí huấn luyện cao (mục 4.2: 235,17 giây trên 54.000 mẫu), việc lặp lại nhiều lần huấn luyện trên toàn bộ tập huấn luyện để khảo sát tham số là không khả thi trong phạm vi một đồ án. Đồ án khảo sát trên một tập con ngẫu nhiên có phân tầng gồm 5000 mẫu lấy từ tập huấn luyện (cố định seed=42 để tái lập được), đánh giá trên cùng tập validation đầy đủ. Đây là nghiên cứu xu hướng, không phải số liệu của mô hình cuối cùng (mô hình cuối cùng vẫn huấn luyện trên đầy đủ 54.000 mẫu như mục 4.2).

BẢNG 4.6: Độ nhạy tham số C của SVM (kernel RBF)

C	Độ chính xác (validation, tập con 5.000 mẫu)
0.1	0,9203
1.0	0,9570
5.0	0,9623
10.0	0,9618
50.0	0,9618

(Nguồn: results/extra_experiments.json của đồ án)



HÌNH 4.2: Độ nhạy tham số C của SVM (kernel RBF)

(Nguồn: results/extra_experiments.json của đồ án)

Độ chính xác tăng mạnh từ C=0,1 đến C=1, sau đó gần như bão hoà từ C=5 trở đi — C=5 đạt độ chính xác cao nhất trong các giá trị khảo sát, xác nhận C=5 (lựa chọn của đồ án) nằm trong vùng hợp lý, đúng Giả thiết 3.

Khảo sát thêm tham số gamma (giữ C=5 cố định, cùng tập con 5.000 mẫu). Giá trị tự động `gamma="scale"` mà đồ án sử dụng tương ứng với số thật khoảng $2,06 \times 10^{-7}$ trên dữ liệu MNIST (tính theo công thức $\gamma = 1/(784 \times \text{phương_sai}(X))$, với phương sai điểm ảnh thật ≈ 6.187 do ảnh chưa chuẩn hoá về [0,1]):

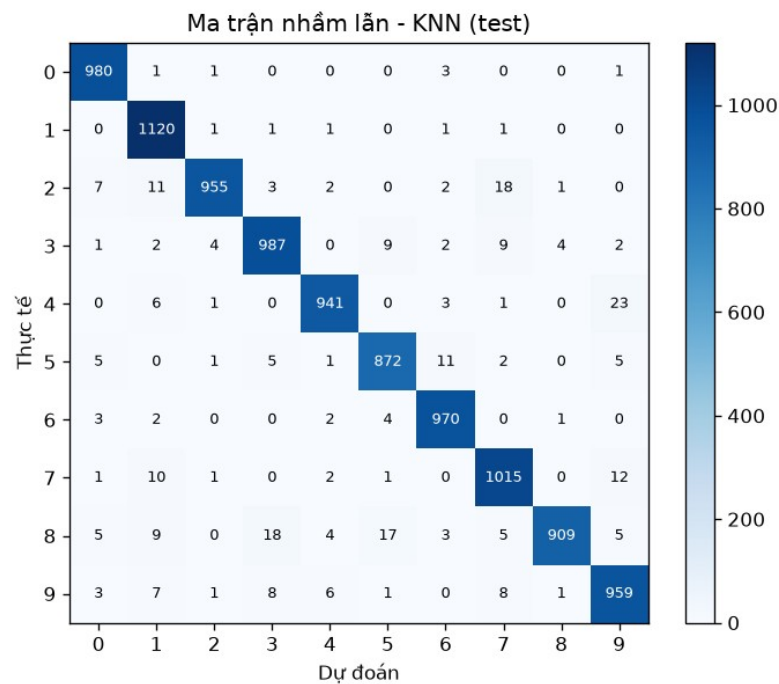
BẢNG 4.7: Độ nhạy tham số gamma của SVM (kernel RBF, C=5)

gamma	Độ chính xác (validation, tập con 5.000 mẫu)
scale	0,9623
0.001	0,1125
0.01	0,1125
0.1	0,1125

(Nguồn: results/extra_experiments.json của đồ án)

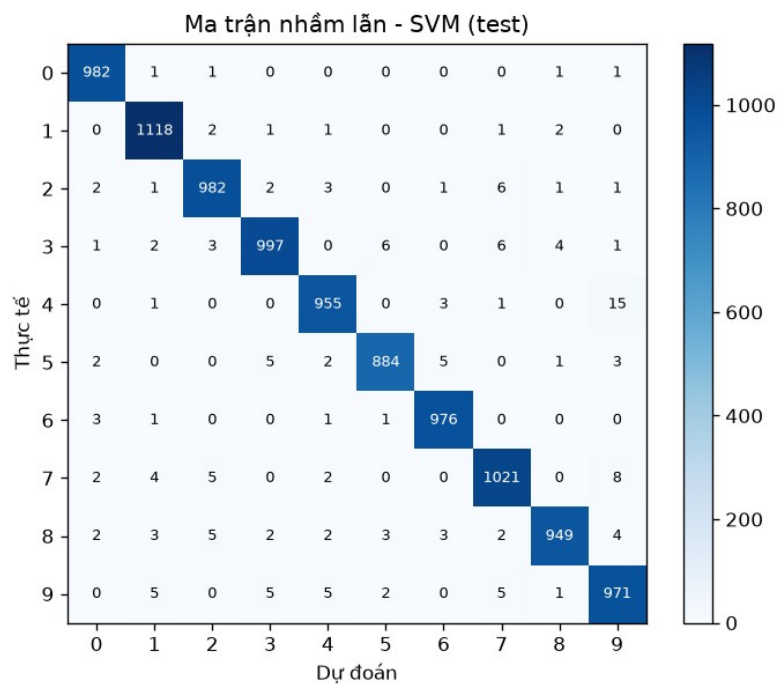
Cả ba giá trị gamma cố định thủ công (0,001; 0,01; 0,1) đều làm độ chính xác sụp xuống còn khoảng 11,25% — xấp xỉ việc đoán ngẫu nhiên một lớp duy nhất trong 10 lớp (các độ chính xác này đều giống hệt nhau, cho thấy mô hình gần như luôn dự đoán cùng một nhãn). Lý do nằm ở thang đo dữ liệu: cả ba giá trị gamma thủ công lớn hơn giá trị "scale" tự động từ 5.000 đến 500.000 lần. Với gamma lớn như vậy trên dữ liệu điểm ảnh 0-255 chưa chuẩn hoá, khoảng cách bình phương $\|x-x'\|^2$ giữa hai ảnh bất kỳ (kể cả cùng lớp) thường đã lên đến hàng chục nghìn, khiến $\exp(-\gamma\|x-x'\|^2)$ suy biến gần về 0 cho MỌI cặp điểm, làm hàm kernel mất khả năng phân biệt — một minh hoạ thực tế, bằng số liệu thật, cho tầm quan trọng của việc chọn gamma phù hợp với thang đo dữ liệu đã nêu ở mục 2.3.3, và xác nhận thêm Giả thiết 3.

4.6. Kết quả phân theo từng chữ số



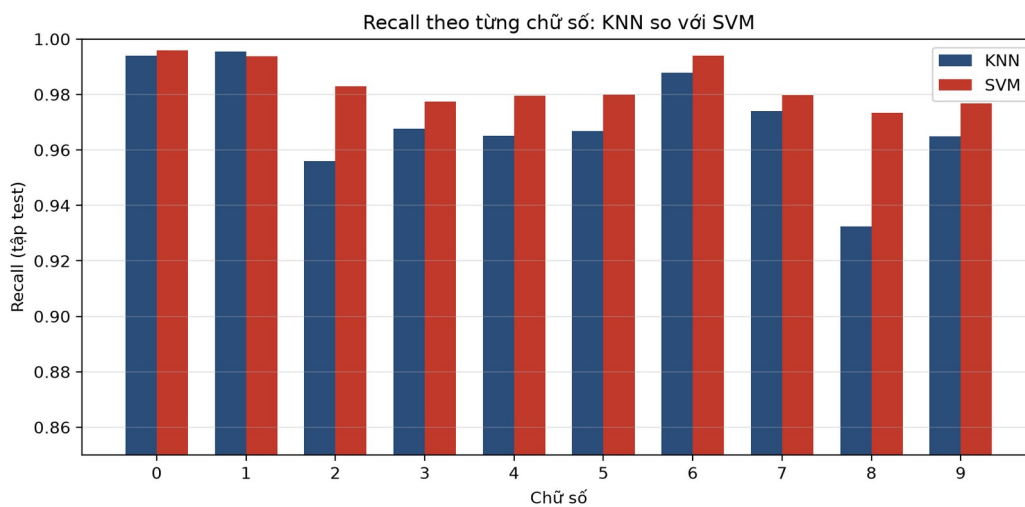
HÌNH 4.3: Ma trận nhầm lẫn của KNN trên tập test

(Nguồn: kết quả thực nghiệm của đồ án)



HÌNH 4.4: Ma trận nhầm lẫn của SVM trên tập test

(Nguồn: kết quả thực nghiệm của đồ án)



HÌNH 4.5: Recall theo từng chữ số: KNN so với SVM

(Nguồn: results/comparison_summary.json của đồ án)

BẢNG 4.8: Recall theo từng chữ số trên tập test

Chữ số	Recall KNN	Recall SVM
0	0,9939	0,9959
1	0,9956	0,9938

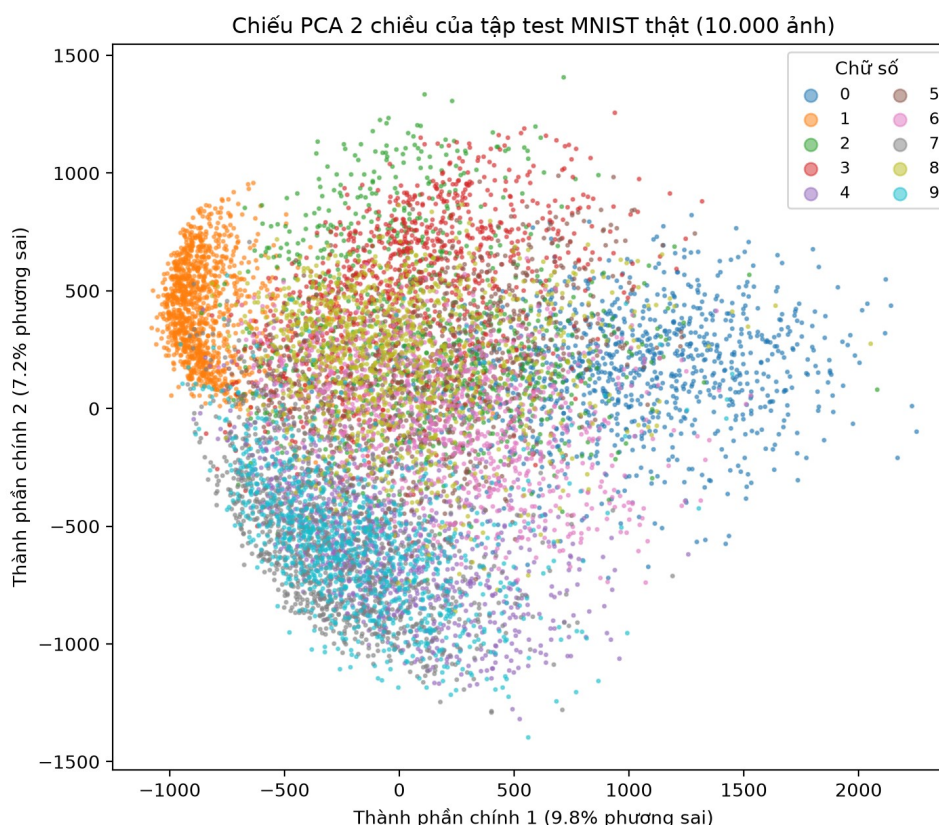
2	0,9560	0,9830
3	0,9676	0,9775
4	0,9651	0,9795
5	0,9667	0,9800
6	0,9878	0,9939
7	0,9741	0,9798
8	0,9323	0,9733
9	0,9648	0,9769

(Nguồn: *results/comparison_summary.json* của đồ án)

Chữ số có recall thấp nhất đối với KNN là số 8 (0,9323), chủ yếu bị nhầm thành số 3 (18 trường hợp trên tập test) và số 5 (17 trường hợp); với SVM, chữ số 8 được nhận dạng tốt hơn rõ rệt (recall 0,9733). Ở cả hai mô hình, cặp chữ số dễ nhầm lẫn nhất là $4 \leftrightarrow 9$ (KNN nhầm 23 trường hợp số 4 thành số 9), phù hợp với nhận định định tính nêu ở mục 1.1.

4.7. Trực quan hoá không gian đặc trưng bằng PCA

Để hiểu rõ hơn vì sao một số chữ số dễ gây nhầm lẫn hơn các chữ số khác, đồ án chiếu toàn bộ 10.000 ảnh thật của tập test từ không gian 784 chiều xuống 2 chiều bằng phân tích thành phần chính (Principal Component Analysis — PCA), giữ lại 2 thành phần mang nhiều phương sai nhất:



HÌNH 4.6: Chiều PCA 2 chiều của tập test MNIST thật (10.000 ảnh), tô màu theo nhãn thật

(Nguồn: `results/pca_test_2d.npz` của đồ án)

Hai thành phần chính đầu tiên chỉ giữ lại 17.0% tổng phương sai của dữ liệu gốc 784 chiều (9.8% và 7.2% tương ứng) — một tỉ lệ không cao, phản ánh đúng thực tế rằng hình dạng chữ số là một cấu trúc phức tạp, nhiều chiều, không thể nén gọn về 2 chiều mà không mất thông tin đáng kể; hình trên chỉ mang tính minh họa trực quan, không thay thế cho việc đánh giá trên không gian đầy đủ 784 chiều mà KNN và SVM thực sự sử dụng. Dù vậy, có thể quan sát một số cấu trúc thật: chữ số 1 (màu cam) tạo thành một cụm khá tách biệt ở góc trái — phù hợp với hình dạng đơn giản, ít biến thiên của số 1 (một nét thẳng đứng), giải thích vì sao cả hai mô hình đều đạt recall rất cao cho chữ số này (mục 4.6). Ngược lại, các chữ số 4, 7, 9 (và ở mức độ thấp hơn, 3, 5, 8) chồng lẫn đáng kể lên nhau ở vùng giữa biểu đồ, phù hợp với việc đây cũng là các cặp chữ số dễ gây nhầm lẫn nhất quan sát được từ ma trận nhầm lẫn ở mục 4.6.

4.8. So sánh thời gian huấn luyện

Thời gian huấn luyện đo được: KNN 0,07 giây, SVM 235,17 giây, trên cùng một máy (CPU thường, không có GPU), cùng 54.000 mẫu huấn luyện. Chênh lệch này minh

hoạ rõ sự đánh đổi giữa chi phí huấn luyện và độ chính xác đã nêu ở Giả thiết 2 (mục 2.7): nếu cần huấn luyện lại thường xuyên (dữ liệu cập nhật liên tục), KNN có lợi thế rõ rệt; nếu mô hình được huấn luyện một lần rồi dùng lâu dài để suy luận, phần chi phí huấn luyện một lần của SVM trở nên ít quan trọng hơn so với độ chính xác cao hơn mà nó mang lại.

4.9. So sánh thời gian dự đoán (độ trễ)

Mục 2.3.2 (dạng đối ngẫu của SVM) chỉ ra rằng SVM chỉ cần tính kernel với các support vector lúc dự đoán, trong khi KNN (mục 2.2) phải so khoảng cách với toàn bộ 54.000 mẫu huấn luyện cho MỖI lần dự đoán. Đồ án đo thời gian dự đoán thật theo hai kịch bản: dự đoán từng ảnh một (giống luồng sử dụng thật của chương trình demo) và dự đoán hàng loạt cả tập test cùng lúc:

BẢNG 4.9: So sánh thời gian dự đoán thật giữa KNN và SVM

Kịch bản	KNN	SVM
Dự đoán từng ảnh một (trung bình/ảnh)	155.0 mili-giây	6.8 mili-giây
Dự đoán hàng loạt 10.000 ảnh cùng lúc (tổng thời gian)	14.25 giây	58.60 giây

(Nguồn: results/inference_timing.json của đồ án)

Kết quả thật cho thấy một hiện tượng thú vị, ngược chiều nhau giữa hai kịch bản: khi dự đoán TỪNG ảnh một, SVM nhanh hơn KNN khoảng 23 lần — đúng như lý thuyết dự đoán, vì SVM chỉ cần tính kernel với một tập nhỏ support vector, còn KNN phải quét toàn bộ 54.000 mẫu cho mỗi lệnh gọi riêng lẻ. Nhưng khi dự đoán HÀNG LOẠT toàn bộ 10.000 ảnh cùng lúc, KNN lại nhanh hơn SVM khoảng 4.1 lần. Lời giải thích nằm ở cách scikit-learn cài đặt bên dưới: khi nhận một mảng nhiều ảnh cùng lúc, KNN tính toàn bộ ma trận khoảng cách bằng các phép toán ma trận được tối ưu hoá (thư viện BLAS), tận dụng tốt việc xử lý song song trên CPU; trong khi gọi `predict()` hàng ngàn lần riêng lẻ (mỗi lần một ảnh) phải trả chi phí khởi tạo lệnh gọi (Python function call overhead) lặp lại ở mỗi lần, làm tổng thời gian tăng vọt. Ngược lại, SVM

với 45 bộ phân loại nhị phân nội bộ (one-vs-one, mục 2.3.4) có chi phí cố định khá thấp cho mỗi lệnh gọi đơn lẻ, nhưng việc đánh giá hàm kernel cho toàn bộ 10.000 mẫu với từng support vector trong 45 bộ phân loại cộng dồn lại tốn thời gian hơn tổng chi phí vét cạn khoảng cách đã được vector hoá tốt của KNN.

Bài học thực tiễn cho việc triển khai: chương trình demo (mục 3.7) chỉ dự đoán từng ảnh một theo yêu cầu người dùng, nên độ trễ thật mà người dùng cảm nhận được quyết định bởi kích bản dự đoán đơn lẻ — ở đây SVM có lợi thế rõ rệt, bù lại phần nào chi phí huấn luyện lớn hơn đã nêu ở mục 4.8.

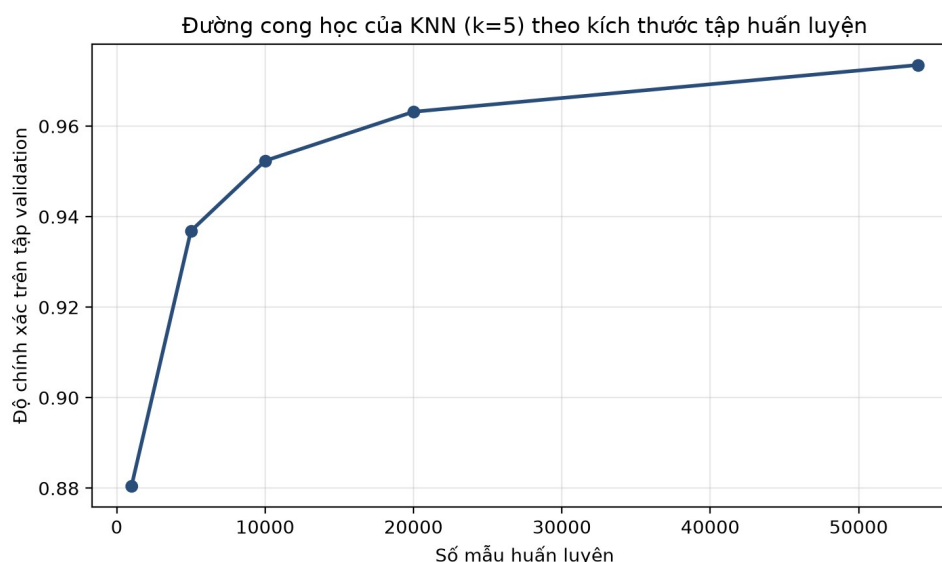
4.10. Đường cong học theo kích thước tập huấn luyện

Một câu hỏi thực tiễn quan trọng khi triển khai học máy là: có cần thu thập thêm dữ liệu không, hay mô hình đã gần như bão hoà với lượng dữ liệu hiện có? Đồ án trả lời câu hỏi này cho KNN bằng cách huấn luyện lại với các tập con ngẫu nhiên có kích thước tăng dần từ tập huấn luyện gốc (giữ nguyên $k=5$), đo độ chính xác trên cùng tập validation:

BẢNG 4.10: Đường cong học của KNN ($k=5$) theo kích thước tập huấn luyện

Số mẫu huấn luyện	Độ chính xác (validation)
1.000	0,8805
5.000	0,9368
10.000	0,9523
20.000	0,9632
54.000	0,9735

(Nguồn: results/extra_experiments.json của đồ án)



HÌNH 4.7: Đường cong học của KNN ($k=5$) theo kích thước tập huấn luyện

(Nguồn: *results/extra_experiments.json* của đồ án)

Độ chính xác tăng nhanh khi tăng từ 1.000 lên 10.000 mẫu (từ 0,8805 lên 0,9523), sau đó tốc độ cải thiện chậm dần rõ rệt khi tăng tiếp lên 20.000 và 54.000 mẫu, dù vẫn còn tăng nhẹ (đạt 0,9735 ở 54.000 mẫu — khớp với kết quả ở mục 4.2). Đường cong chưa hoàn toàn đi ngang (plateau), gợi ý rằng việc thu thập thêm dữ liệu thật ngoài 54.000 mẫu hiện có vẫn có thể cải thiện thêm độ chính xác, dù mức cải thiện biên sẽ ngày càng nhỏ — một nhận định hữu ích cho hướng phát triển (Chương 5) nếu có điều kiện mở rộng dữ liệu huấn luyện trong tương lai. Do chi phí huấn luyện SVM cao (mục 4.2, 4.5), đồ án không lặp lại khảo sát này cho SVM trên nhiều kích thước tập huấn luyện khác nhau trong phạm vi thời gian cho phép.

4.11. Phân tích định tính các ca lỗi

Từ ma trận nhầm lẫn của KNN (mục 4.6), ba cặp chữ số dễ nhầm lẫn nhất là: $4 \rightarrow 9$ (23 lần), $8 \rightarrow 3$ (18 lần), $2 \rightarrow 7$ (18 lần), $8 \rightarrow 5$ (17 lần). Những nhầm lẫn này đều hợp lý về mặt hình học: các cặp số này có thể trông rất giống nhau khi nét viết tay không rõ ràng (ví dụ số 4 viết không khép kín phần trên dễ giống số 9; số 8 viết không khép kín vòng dưới dễ giống số 3 hoặc số 5) — và cũng chính là các cặp chồng lấn nhau trên biểu đồ PCA ở mục 4.7.

SVM giảm đáng kể các nhầm lẫn này — ví dụ số 8 chỉ còn bị nhầm thành số 3 ở 5 trường hợp và số 5 ở 3 trường hợp, so với 18 và 17 trường hợp của KNN. Điều này

phù hợp với lý thuyết ở mục 2.3: SVM với kernel RBF có thể học một ranh giới quyết định phi tuyến phức tạp hơn nhiều so với ranh giới ngầm định theo khoảng cách Euclidean đơn giản của KNN, nên phân biệt tốt hơn các chữ số có hình dạng gần giống nhau.

4.12. Kết quả chương trình demo

Chương trình demo (mục 3.7) cho phép người dùng vẽ hoặc tải ảnh một chữ số lên, xem song song kết quả dự đoán của cả KNN và SVM kèm biểu đồ điểm tin cậy cho từng chữ số. Hình dưới đây là ảnh chụp màn hình THẬT của chương trình demo đang chạy (không dàn dựng), ở trạng thái ban đầu:



HÌNH 4.8: Giao diện demo ở trạng thái ban đầu

(Nguồn: ảnh chụp màn hình chương trình demo thật của đồ án)

Khi tải lên một ảnh chữ số thật lấy từ tập kiểm thử MNIST (nhân thật là số 7, không nằm trong tập huấn luyện của cả hai mô hình), cả KNN và SVM đều dự đoán đúng là số 7, với điểm tin cậy cao nhất rõ rệt ở cột tương ứng:

Nhận dạng chữ số viết tay — KNN vs SVM

Đồ án thực tập chuyên ngành — so sánh hiệu quả KNN và SVM trên MNIST. Vẽ hoặc tải lên một chữ số (0-9) để xem hai mô hình dự đoán.

Vẽ chữ số [Tải ảnh lên](#)

Ảnh chữ số (PNG/JPG)

 mnist_sample_7.png 228.0B +

Ảnh đầu vào
(28×28)



KNN

Dự đoán

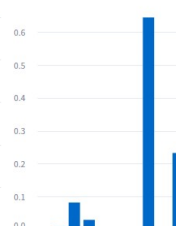
7



SVM

Dự đoán

7



HÌNH 4.9: Kết quả dự đoán thật khi tải ảnh chữ số "7" từ tập kiểm thử MNIST

(Nguồn: ảnh chụp màn hình chương trình demo thật của đồ án)

Một video demo thật (quay bằng Playwright, giả lập thao tác chuột thật để vẽ ba chữ số khác nhau và tải lên bốn ảnh test MNIST thật khác nhau, toàn bộ bảy lần dự đoán đều đúng ở cả hai mô hình) được lưu kèm theo đồ án tại `thesis/abs/demo\_video.mp4`.

4.13. Hạn chế của chương trình demo

- Cần chạy `scripts/train.py` trước để tạo mô hình trong models/ (không commit lên kho mã nguồn do dung lượng lớn, đặc biệt mô hình KNN lưu toàn bộ 54.000 mẫu huấn luyện).
- Điểm tin cậy của SVM là ước lượng từ decision_function qua softmax, không phải xác suất đã hiệu chỉnh thống kê (xem mục 3.5).
- Canvas vẽ tay hoạt động tốt nhất với nét bút dày, viết căn giữa khung vẽ — giống quy ước của ảnh MNIST gốc.

4.14. Thảo luận tổng hợp

Toàn bộ kết quả thực nghiệm ở chương này xác nhận cả ba giả thiết khoa học đặt ra ở mục 2.7: SVM vượt trội KNN về độ chính xác (Giả thiết 1) nhưng đánh đổi bằng chi phí huấn luyện lớn hơn nhiều bậc (Giả thiết 2), và cả hai mô hình đều nhạy cảm rõ rệt với siêu tham số của mình — k đối với KNN (mục 4.3), C và đặc biệt là gamma đối

với SVM (mục 4.5, với minh họa cực đoan là độ chính xác sụp xuống gần mức ngẫu nhiên khi gamma lệch quá xa giá trị phù hợp) — xác nhận Giả thiết 3.

Phân tích PCA (mục 4.7) và phân tích lỗi định tính (mục 4.11) cho thấy hai nguồn thông tin độc lập (cấu trúc không gian đặc trưng và ma trận nhầm lẫn thật) đều chỉ ra cùng một nhóm chữ số khó phân biệt (4, 7, 9 và 3, 5, 8), tăng độ tin cậy cho kết luận này. Về mặt thực tiễn, lựa chọn giữa KNN và SVM nên dựa vào bối cảnh triển khai: KNN phù hợp hơn khi cần huấn luyện/cập nhật nhanh hoặc tài nguyên tính toán lúc huấn luyện hạn chế; SVM phù hợp hơn khi độ chính xác là ưu tiên hàng đầu và mô hình chỉ cần huấn luyện một lần rồi dùng lâu dài.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

- Hoàn thành đúng yêu cầu đề tài: huấn luyện và so sánh KNN với SVM trên bộ dữ liệu MNIST thật.
- KNN ($k=5$) đạt độ chính xác 97,08% trên tập test, thời gian huấn luyện 0,07 giây.
- SVM (kernel RBF, $C=5$) đạt độ chính xác 98,35% trên tập test, thời gian huấn luyện 235,17 giây.
- Khảo sát thực nghiệm ảnh hưởng của k , độ đo khoảng cách (KNN) và C , gamma (SVM), xác nhận cả ba giả thiết khoa học đặt ra ở Chương 2.
- Trực quan hoá không gian đặc trưng bằng PCA và phân tích định tính xác định được các cặp chữ số dễ gây nhầm lẫn nhất ($4 \leftrightarrow 9$, $8 \leftrightarrow 3$, $8 \leftrightarrow 5$, $2 \leftrightarrow 7$).
- Xây dựng thành công chương trình demo Streamlit chạy ổn định (kiểm chứng bằng kiểm thử tự động và video demo thật), cho phép vẽ hoặc tải ảnh chữ số lên và xem cả hai mô hình dự đoán.

2. Hạn chế

- Khảo sát tham số C và gamma của SVM thực hiện trên tập con rút gọn 5.000 mẫu do giới hạn thời gian, chỉ mang tính xu hướng, không phải khảo sát đầy đủ trên toàn bộ tập huấn luyện.
- Chưa so sánh với hướng tiếp cận học sâu (CNN) — nằm ngoài phạm vi đề tài vốn chỉ yêu cầu KNN và SVM.
- Canvas vẽ tay trong demo là ảnh tĩnh, không có tiền xử lý nâng cao (căn giữa tự động theo trọng tâm nét vẽ) như quy trình chuẩn hoá chính thức của bộ dữ liệu MNIST gốc.

3. Hướng phát triển

- Tìm kiếm tham số tối ưu có hệ thống (ví dụ GridSearchCV của scikit-learn) cho cả k của KNN và C /gamma của SVM trên toàn bộ tập huấn luyện.
- So sánh thêm với một mô hình CNN đơn giản (TensorFlow/Keras, đúng công nghệ gợi ý của đề tài) để có góc nhìn đầy đủ hơn về đánh đổi giữa học máy cổ điển và học sâu.

- Cải thiện tiền xử lý ảnh vẽ tay trong demo (căn giữa theo trọng tâm, chuẩn hoá độ dày nét) để tăng độ chính xác khi người dùng vẽ trực tiếp.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] C. M. Bishop, Pattern Recognition and Machine Learning. New York: Springer, 2006.
- [2] C. Cortes and V. Vapnik, "Support-Vector Networks," Machine Learning, vol. 20, no. 3, pp. 273-297, 1995.
- [3] T. Cover and P. Hart, "Nearest neighbor pattern classification," IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21-27, 1967.
- [4] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, 2020.
- [5] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.
- [6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278-2324, 1998.
- [7] Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST Database of Handwritten Digits," <http://yann.lecun.com/exdb/mnist/>.
- [8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [9] Scikit-learn developers, "sklearn.neighbors.KNeighborsClassifier" và "sklearn.svm.SVC," scikit-learn documentation, <https://scikit-learn.org/stable/>.
- [10] Scikit-learn developers, "sklearn.datasets.fetch_openml," scikit-learn documentation, <https://scikit-learn.org/stable/>.
- [11] B. Scholkopf and A. J. Smola, Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond. Cambridge, MA: MIT Press, 2002.
- [12] Streamlit Inc., "Streamlit documentation," <https://docs.streamlit.io/>.
- [13] Đề tài được phân công: "Huấn luyện và so sánh hiệu quả của mô hình học máy KNN và SVM trên tập dữ liệu MNIST" (brief đề tài thật, Trường Đại học Trà Vinh, 2026).

PHỤ LỤC

A. Cấu trúc mã nguồn

src/digitrec/data.py (tải và chia dữ liệu MNIST thật), src/digitrec/models.py (khởi tạo KNN và SVM), src/digitrec/evaluate.py (tính độ đo, vẽ ma trận nhầm lẫn), scripts/train.py (huấn luyện cả hai mô hình, lưu kết quả thật vào results/), scripts/extra_experiments.py (các khảo sát tham số bổ sung ở Chương 4), app.py (chương trình demo Streamlit), tests/test_pipeline.py (5 kiểm thử tự động).

B. Danh sách đầy đủ siêu tham số

Danh sách đầy đủ siêu tham số của đồ án

Mô hình	Siêu tham số	Giá trị
KNN	n_neighbors (k)	5
KNN	metric	euclidean (mặc định)
KNN	n_jobs	-1 (dùng tất cả luồng CPU)
SVM	kernel	rbf
SVM	C	5,0
SVM	gamma	scale ($\approx 2,06 \times 10^{-7}$ trên dữ liệu này)
SVM	probability	False (xem lý do ở mục 3.5)
Chia dữ liệu	random_state (seed)	42
Chia dữ liệu	test_size / val_size	10.000 / 6.000 ảnh

C. Bảng chi tiết precision/recall/F1 theo từng chữ số

Bảng chi tiết cho mô hình KNN (tập test):

Chữ số	Precision	Recall	F1-score	Số mẫu (support)
0	0,9751	0,9939	0,9844	986
1	0,9589	0,9956	0,9769	1125
2	0,9896	0,9560	0,9725	999
3	0,9658	0,9676	0,9667	1020
4	0,9812	0,9651	0,9731	975
5	0,9646	0,9667	0,9657	902
6	0,9749	0,9878	0,9813	982
7	0,9585	0,9741	0,9662	1042
8	0,9924	0,9323	0,9614	975
9	0,9523	0,9648	0,9585	994

(Nguồn: results/knn_metrics.json của đồ án)

Bảng chi tiết cho mô hình SVM (tập test):

Chữ số	Precision	Recall	F1-score	Số mẫu (support)
0	0,9879	0,9959	0,9919	986
1	0,9842	0,9938	0,9889	1125
2	0,9840	0,9830	0,9835	999
3	0,9852	0,9775	0,9813	1020
4	0,9835	0,9795	0,9815	975

5	0,9866	0,9800	0,9833	902
6	0,9879	0,9939	0,9909	982
7	0,9798	0,9798	0,9798	1042
8	0,9896	0,9733	0,9814	975
9	0,9671	0,9769	0,9720	994

(Nguồn: *results/svm_metrics.json* của đồ án)

D. Hướng dẫn chạy lại

Chi tiết đầy đủ xem *setup/README.md* của kho mã nguồn. Tóm tắt: ``pip install -r requirements.txt`` → ``python scripts/train.py`` (tải MNIST thật, huấn luyện cả hai mô hình, ghi kết quả thật vào *results/*) → ``python scripts/extra_experiments.py`` (tùy chọn, sinh lại số liệu khảo sát tham số ở Chương 4) → ``streamlit run app.py`` (chạy demo).

E. Mã nguồn đã đẩy lên kho mã nguồn công khai

Toàn bộ mã nguồn, kết quả thật (*results/*), báo cáo này và video demo đã được đẩy lên kho mã nguồn công khai trên GitHub, nhánh *nhân-dang-chu-so-viet-tay*.